

Blockchain Honor Minor Degree : Sem VIII

HBCC801 : Decentralized Finance (DeFi)

**Module - 2: What is Decentralized Finance (DeFi) ?
(4 Hours)**

Instructors : Lifna C S

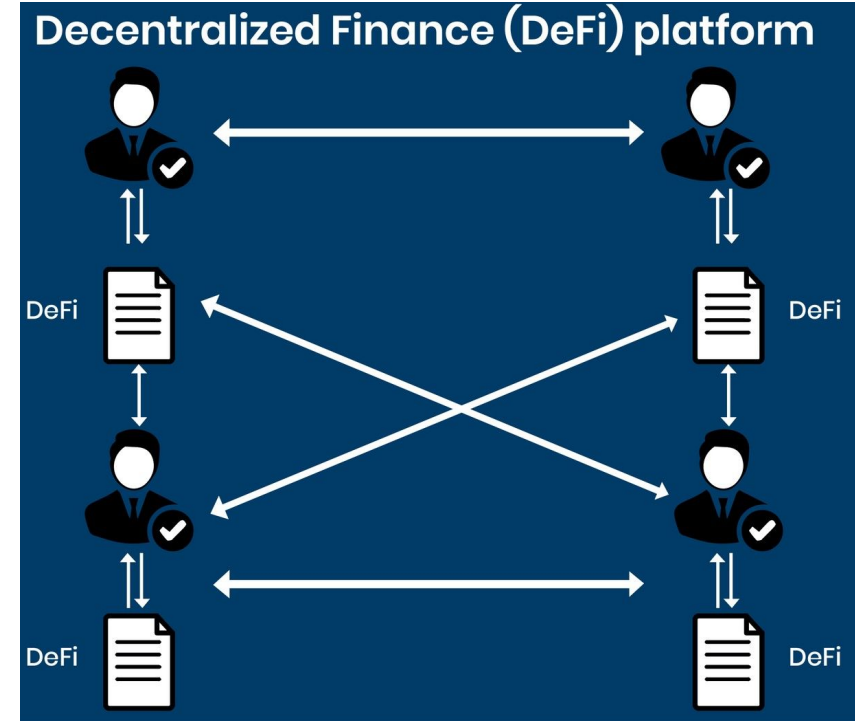
Department of Computer Engineering, VESIT, Mumbai

Topics to be covered

1. What is DeFi ?
2. The DeFi Ecosystem
3. Problems that DeFi Solves
4. How Decentralized is DeFi?
5. Defi key Categories:-
 - a. Stablecoins,
 - b. Lending and Borrowing
 - c. Derivations
 - d. Lottery
 - e. Insurance
 - f. Stable coin and pegging
 - g. Exchanges
 - h. Fund Management
 - i. Payments
6. How Decentralized Finance Could Make Investing More Accessible ?

1. What is DeFi (Decentralized Finance) ?

- Decentralized Finance (DeFi) enables financial services without centralized intermediaries
- Built using Decentralized Applications (DApps)
- Mostly deployed on Ethereum blockchain
- Replaces traditional institutions like:
 - Banks
 - Exchanges
 - Insurance
 - Money markets
- Known as “Money Legos” due to composability

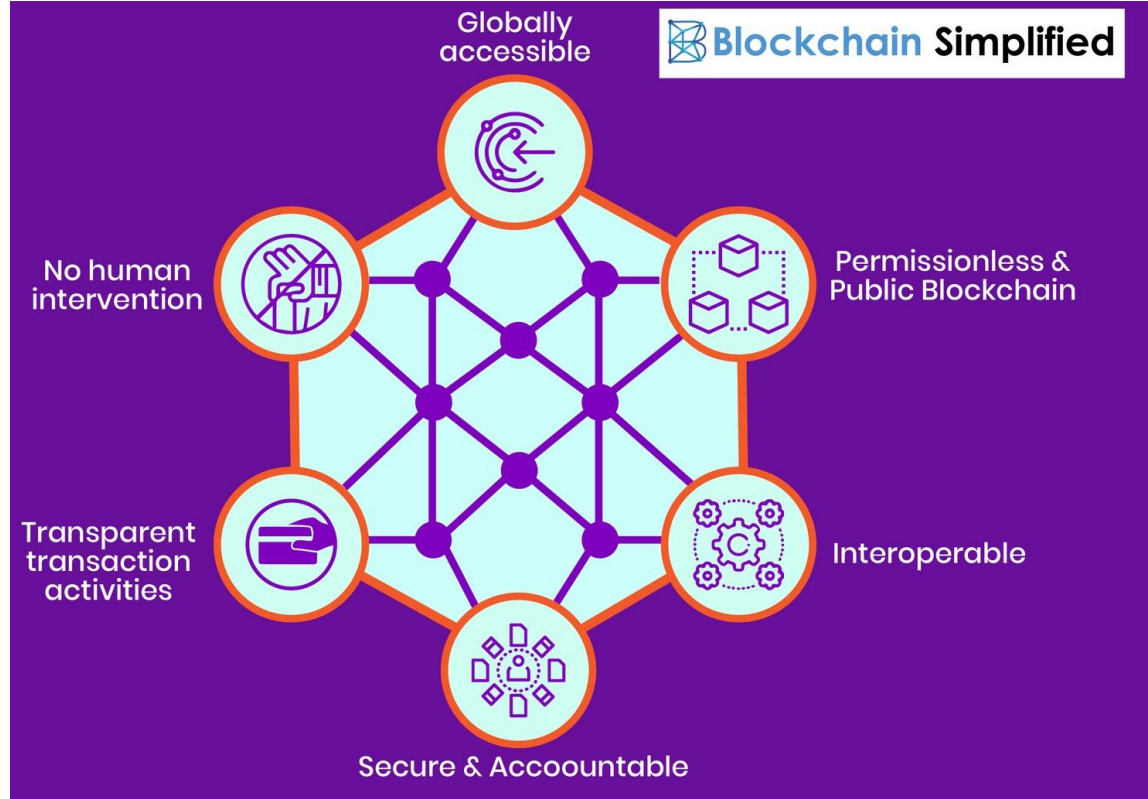


Courtesy :

- [DeFi - The Future of Finance, Campbell et. al](#)
- [Blockchain Simplified](#)

1. What is DeFi (Decentralized Finance) ?

DeFi Benefits:

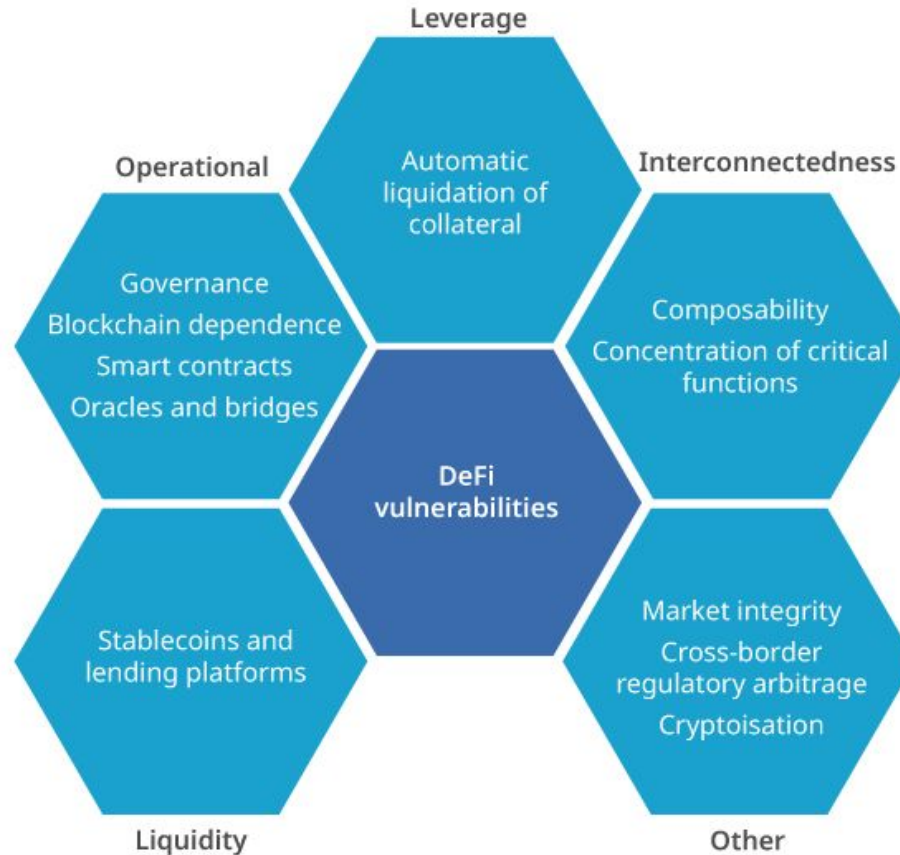


Courtesy :

- [DeFi - The Future of Finance, Campbell et. al](#)
- [Blockchain Simplified](#)

1. What is DeFi (Decentralized Finance) ?

DeFi Vulnerabilities :



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [Bank of International Settlements](#)

2. The DeFi Ecosystem

Core Components:

- Blockchain (Ethereum)
- Smart Contracts
- DApps
- Users & Liquidity Providers

Key Feature:

- Assets are locked as collateral in smart contracts
- Measured as **Total Value Locked (TVL)**

Insight:

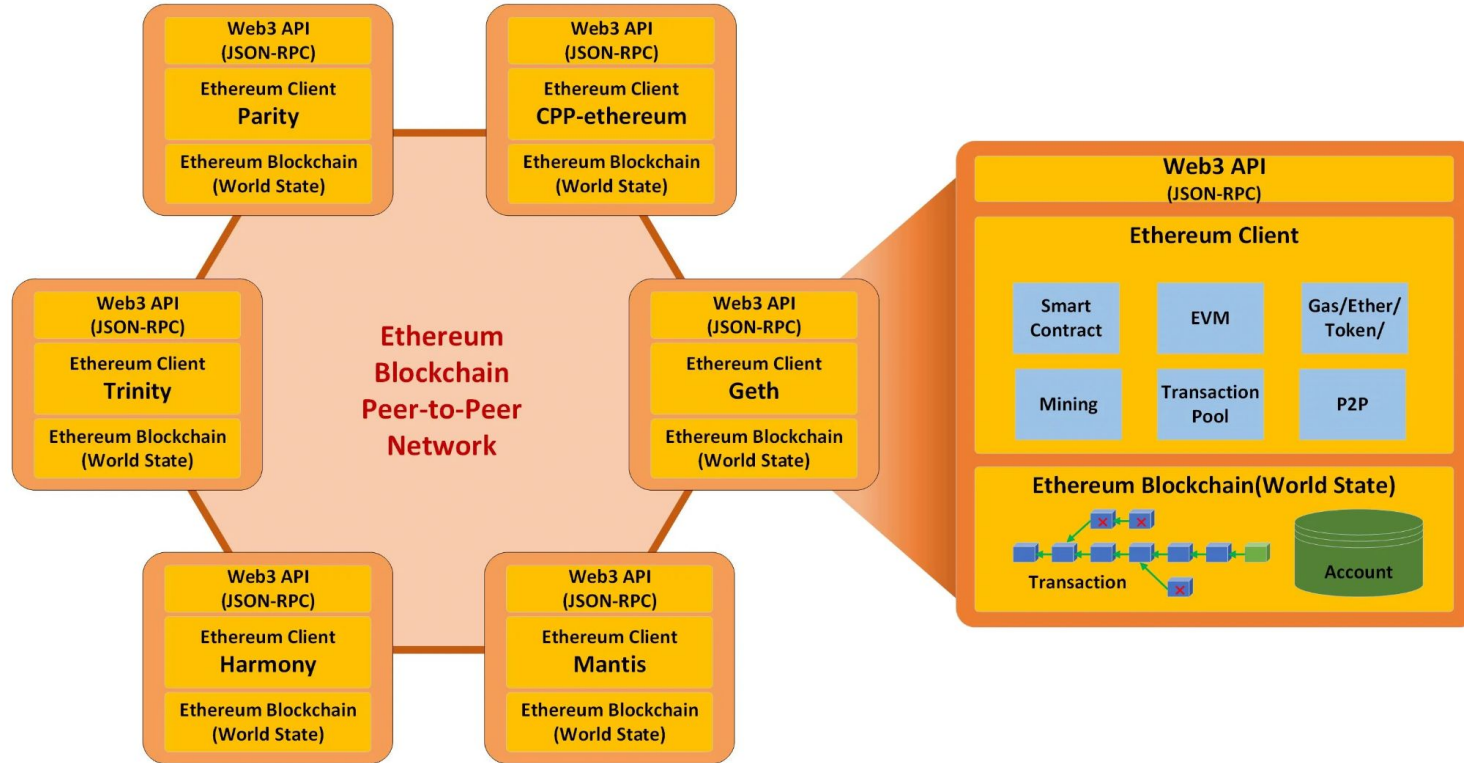
- Rapid growth of TVL indicates strong DeFi adoption

2. The DeFi Ecosystem - What is Ethereum?

- A [Blockchain network](#) that introduced a built-in **Turing-complete programming language** that can be **used for creating** various decentralized applications(also called **Dapps**).
- Ethereum network is fueled by its own **cryptocurrency called 'ether'**.
- Allow the implementation of **smart contracts**. Smart contracts can be thought of as 'cryptographic bank lockers' which contain certain values.
- These cryptographic lockers can only be unlocked when certain conditions are met.
- Ethereum is a network that can be applied to various other sectors.
- called **Blockchain 2.0** → it proved that blockchain can be used beyond the financial sector.
- The consensus mechanism used in Ethereum is [Proof of Stakes\(PoS\)](#), which is **more energy efficient** than, [Proof of Work\(PoW\)](#).
- PoS depends on the **amount of stake a node holds**.



2. The DeFi Ecosystem - Ethereum Architecture



Courtesy :
[DeFi - The Future of Finance, Campbell et. al](#)
[Packt - Ethereum Architecture](#)

2. The DeFi Ecosystem - Features of Ethereum

1. Smart contracts:
 - a. Ethereum allows the creation and deployment of smart contracts.
 - b. Smart contracts are created mainly using a programming language called **solidity**.
2. Ethereum Virtual Machine (EVM):
 - a. **a runtime environment for compiling and deploying Ethereum-based smart contracts.**
3. Ether:
 - a. **cryptocurrency of the Ethereum network.**
 - b. **only acceptable form of payment for transaction fees** on the Ethereum network.
4. Decentralized applications (Daaps):
 - a. **has its backend code running on a decentralized peer-to-peer network.**
 - b. Has a frontend and UI to make calls and query data from its backend.
 - c. They **operate on Ethereum** and **perform the same function irrespective of the environment** in which they get executed.
5. Decentralized autonomous organizations (DAOs):
 - a. **works in a democratic and decentralized** fashion.
 - b. **relies on smart contracts for decision-making** within the organization.



2. The DeFi Ecosystem - Real-World Applications of Ethereum

1. Voting:

- a. Voting systems are **adopting Ethereum**.
- b. The **results of polls are available publicly, ensuring a transparent fair system thus eliminating voting malpractices.**

2. Agreements:

- a. With Ethereum smart contracts, agreements and contracts **can be maintained and executed without any alteration.**
- b. Ethereum can be **used for creating smart contracts and for digitally recording transactions based on them.**

3. Banking systems:

- a. **Due to the decentralized nature of the Ethereum blockchain it becomes challenging for hackers to gain unauthorized access to the network.**
- b. **Makes payments on the Ethereum network secure**

4. Shipping:

- a. **Ethereum provides a tracking framework that helps with the tracking of cargo and prevents goods from being misplaced.**

5. Crowdfunding:

- a. **helps to increase trust and information symmetry.**
- b. **It creates many possibilities for startups by raising funds to create their own digital cryptocurrency.**



2. The DeFi Ecosystem - Components of Ethereum Network

1. Ethereum Node
2. Ethereum Client
3. Ether
4. Gas
5. Ethereum Accounts
6. Nonce
7. Storage Root
8. Ethash
9. Ethereum Virtual Machine (EVM)
10. Transactions



2. The DeFi Ecosystem - Ethereum

Components of Ethereum Network - (1) Ethereum Node

- a computer that is running the software client.
- Nodes communicate with one another in order to validate transactions and record data about the status of the blockchain.
- Responsible for storing, validating, and trading data.
- Each node keeps its own copy of the blockchain and strives to verify that it matches the copies of all the other nodes.
- Every node on the network must process any action that requires a new block to be added to the blockchain.
- This network of continually communicating nodes allows us to avoid relying on a single source of truth and all of the challenges it entails.
- A new block is added based on whether or not the majority of nodes accept it.



Components of Ethereum Network - (1) Ethereum Node (Types)

1. Full Node:

- verify and validate each and every transaction that takes place inside the network
- maintain the state / a full copy of the blockchain.
- When a smart contract transaction occurs,
 - Full nodes also execute all of the instructions in the smart contract.
 - It determines whether or not the smart contract execution is producing the expected results.
- It keeps receiving copies of the entire blockchain including its transactions which are stored locally and keeps the latest state of transaction with itself.
- Eg: Person A performs a transaction to person B,
 - transaction is added to the blockchain,
 - Full nodes verify whether the transaction complies with all the Ethereum specifications,
 - Maintain the latest state of the blockchain by storing or removing the specification if it does not comply.
- Example of a discarder transaction is when a person transfers X ETH to another person but their account contains less ETH.



Components of Ethereum Network - (1) Ethereum Node (Types)

2. Archive Nodes:

- complete nodes that have the “archive mode” option enabled.
- While a **Full Node only stores the latest state of the transaction**,
- the Archive nodes hold all of the blockchain's history data dating back to the genesis block.
- used when blocks prior to the latest 128 blocks are required.
- Eg : using functions like **eth_getBalance** of a historic address would require an archive node, as will interacting with smart contracts launched far earlier in the blockchain.
- Archive Nodes memory requirement :
 - **require more than 6 Terabytes of space**, contrary to Full node which only requires a little over 500 Gigabytes of disk space.
 - **are not useful for average people**,
 - they are effective in the application of block exploring, wallet vending, and chain analytics.



Components of Ethereum Network - (1) Ethereum Node (Types)

3. Light Nodes:

- [does not hold the complete current blockchain state](#)
- stores only the block header.
- suitable for low memory and computational devices since maintaining a light node involve the least investment in hardware, running costs, and technical skill.
- Light nodes [rely on full nodes to function.](#)
- These nodes **do not need to run continuously or read and publish a large amount of data on the blockchain.**
- It provides [an easy way to create a wallet, especially for beginners.](#)
- Eg : **Solid-State Drives (SSD)** cannot afford to store the gigabytes of data that other nodes take.
- But there are some limitations of light nodes which cannot be denied, [there is no guarantee that the light wallet provider will be online when it is needed.](#)



Components of Ethereum Network - (2) Ethereum Client

- **software program** that is used to implement the Ethereum specification
- **connect itself with other Ethereum clients** over a peer-to-peer network.
- Different Ethereum clients can communicate with one another if they follow the reference specification and the defined communication protocols.
- These interactions among different clients in the network **take place using various programming languages**
 - like Geth (Go), OpenEthereum (Rust),
 - Nethermind (C#, .NET), Besu (Java),
 - Erigon (Go/Multi).
- The **yellow paper** is the Ethereum protocol that allows anybody to run a client to construct a node.
- **Ethereum sets standard behaviors that all Ethereum clients must adhere to**
- **Ethereum's specs** enabled the blockchain to allow for different, but interoperable, software implementations of an Ethereum client by providing standard documentation and simple language.



Components of Ethereum Network - (2) Ethereum Client (Types)

1. Full Client:

- save the complete Ethereum blockchain,
- which **might take several days to synchronize** and
- takes a **massive amount of disc space** – more than 1 Terabyte, according to the most recent estimates.
- **Enable connected nodes to conduct all network functions**, including as
 - mining,
 - transaction
 - block-header validation,
 - smart contract execution.



Components of Ethereum Network - (2) Ethereum Client (Types)

2. Light Client:

- do not always need to necessarily keep all of the data,
- when data storage and performance are concerns, developers utilize the “light clients”.
- Light clients **provide a portion of full client capability.**
- they can provide quick delivery and free up data storage space.
- The functionality of a light client is adapted to the purposes of the Ethereum client.
- **widely used within wallets to maintain private keys and Ethereum addresses.**
- **manage smart contract interactions and transaction broadcasts.**
- **useful for web3 instances** within JavaScript objects, Dapp browsers
- **obtaining the exchange rate data.**



Components of Ethereum Network - (2) Ethereum Client (Types)

3. Remote Client:

- A remote client is much like a light client.
- The primary distinction is that a remote client **does not keep its own copy of the blockchain or validate transactions or block headers.**
- Remote clients, on the other hand, rely entirely on a full or light client to have access to the Ethereum blockchain network.
- These clients are mostly used as wallets for transmitting and receiving transactions.

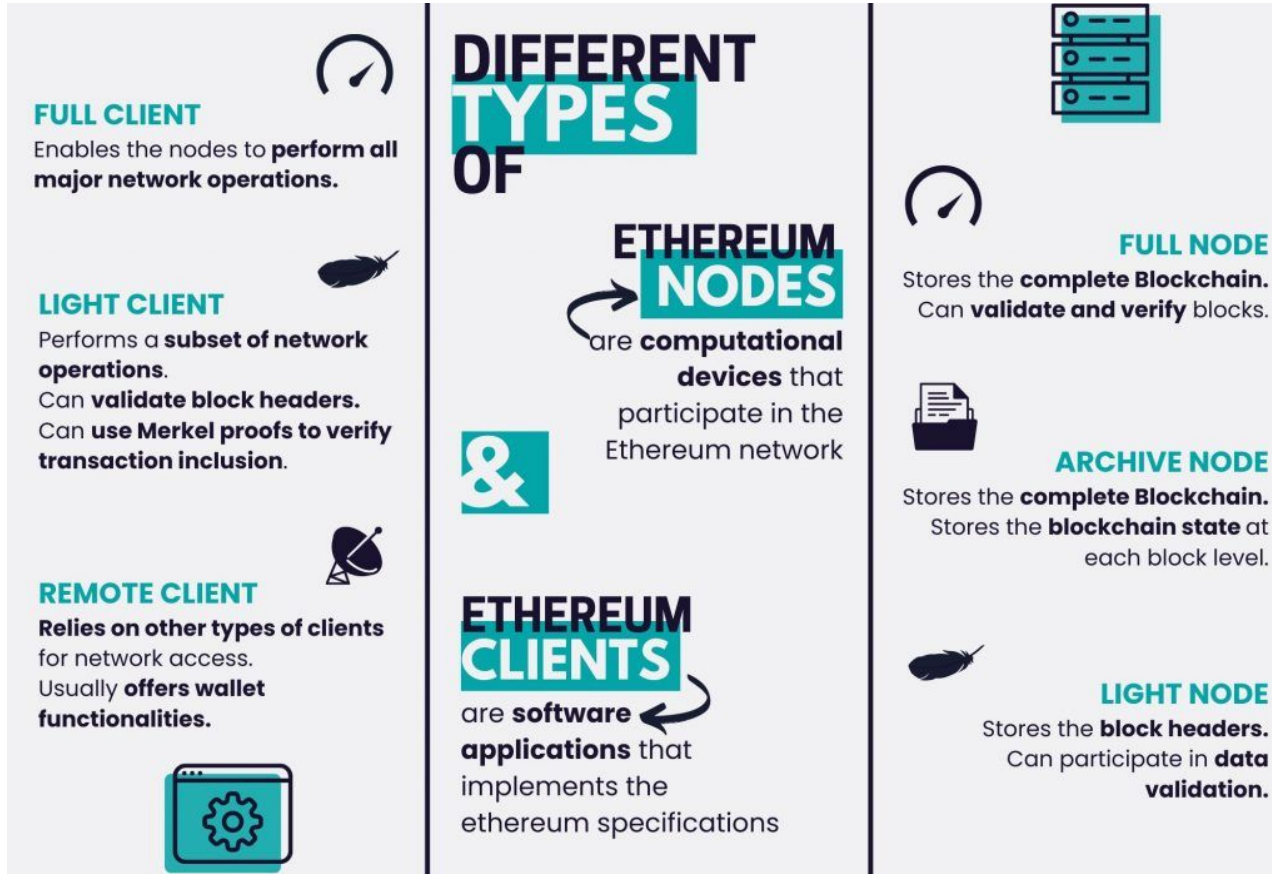


Components of Ethereum Network - Node Vs Client

Ethereum Node	Ethereum Client
A machine running Ethereum client software is referred to as an “Ethereum Node”	A client is an Ethereum implementation that validates all transactions in each block, ensuring the network’s security and data accuracy
The three types of Ethereum Nodes are Full, Light, Archive, and Miner Nodes.	The three types of Ethereum Clients are Full, Light, and Remote Clients
The Ethereum node operating system allows us to access the internet	The Ethereum client computer allows a user to access the node operating system



Components of Ethereum Network - Node Vs Client



Components of Ethereum Network -

	Pros	Cons
Light nodes	• Portable • Resource-efficient • User-friendly	• Do not validate the network • Do not propagate blocks • Do not maintain consensus • Less secure
Full nodes	• Validate the network • Propagate blocks • Maintain consensus • More secure	• Resource-heavy • Harder to maintain • Less user-friendly
Pruned	Flexible storage	Need to revalidate old blocks
Archive	Carry full history	Resource and storage heavy
Mining	• Easily trackable involvement • Can pool with others to increase reward rate	• High and wasteful energy consumption • High equipment cost and barrier to entry
Staking	• Low barrier to entry • Low energy consumption	• Reward system based on luck • Low transparency in staking pools
Masternodes	• Balanced network benefits and rewards • Lower maintenance costs	• High initial investment • Difficult setup process



Components of Ethereum Network - (3) Ether

- type of cryptocurrency used in the Ethereum network
- It is a peer-to-peer currency
- It **tracks and promotes each transaction** in the network.
- It is the second-largest cryptocurrency in the world.
- Other cryptocurrencies can be used to get ether tokens, but vice versa is not true.
 - It means that **ether tokens can't be interchanged by other cryptocurrencies** to render computing power for Ethereum transactions.
- paid as a commission for any execution that affects the state in Ethereum.
- used in the Ethereum algorithm as an incentive for miners to blocks to the blockchain using PoW
- only currency that can be used to pay transaction costs, which go to miners as well.
- Aside from paying for transactions, **ether is often used to purchase gas**, which is used to pay for the computation of any transaction on the Ethereum network.



Components of Ethereum Network - (4) Gas

- An internal currency of the Ethereum network.
- We need gas to run applications on the Ethereum network, much as we need gas to run a vehicle.
- To complete every transaction on the Ethereum network, a consumer must first make a payment—send out ethers—and the intermediate monetary value is known as gas.
- Gas is a unit of measurement on the Ethereum network for the computing power used to execute a smart contract or a transaction.
- The price of gas is very low compared to Ether.
- The execution and resource utilization costs are predetermined in Ethereum in terms of Gas units, called gwei.



Components of Ethereum Network - Ether Denominations

Value (in wei)	Exponent	Common Name	SI Name
1	1	wei	wei
1,000	10^3	babbage	kilowei or femtoether
1,000,000	10^6	lovelace	megawei or picoether
1,000,000,000	10^9	shannon	gigawei or nanoether
1,000,000,000,000	10^{12}	szabo	microether or micro
1,000,000,000,000,000	10^{15}	finney	milliether or milli
<i>1,000,000,000,000,000,000</i>	<i>10^{18}</i>	<i>ether</i>	<i>ether</i>
1,000,000,000,000,000,000,000	10^{21}	grand	kiloether
1,000,000,000,000,000,000,000,000	10^{24}		megaether



Components of Ethereum Network - (5) Ethereum Accounts

- similar to a bank account,
- but for ethers or ETH, where Ethereum can be held, transferred to other accounts.
- can also be **used to execute smart contracts.**
- An entity that is composed of **an Ethereum address along with a private key.**
 - The first 20 bytes of the SHA3 hashed public key is the Ethereum address.
- Ethereum has two types of accounts:
 - **Externally owned account (EOA):**
 - **Contract Account:**



Components of Ethereum Network - (5) Ethereum Accounts (Types)

Ethereum has two types of accounts:

1. Externally owned account (EOA):

- controlled by private keys.
- Each EOA **has a public-private key pair.**
- The **users can send messages by creating and signing transactions.**

Advantages of EOA

1. Transactions from an external account to a contract account can trigger code that can execute many different actions, such as transferring tokens or even creating a new contract.
2. Externally Owned Accounts cannot list incoming transactions.



2. Contract Account:

- **controlled by contract codes.**
- These **codes are stored with the account.**
- Each contract account **has an ether balance associated with it.**
- The contract code of these accounts **gets activated every time a transaction from an EOA or a message from another contract is received by it.**
- When the contract code activates, **it allows to read/write the message to the local storage, send messages and create contracts.**
- **Types of Contract Accounts :**
 - i. **Simple Account:** The account is *created and owned by a single account holder.*
 - ii. **Multisig (multisignature) Account:** A Multisig Wallet *contains several owner Accounts,* one of which is also the creator Account.



Components of Ethereum Network - (5) Ethereum Accounts (Types)

Advantages of CA:

1. A contract account **can list incoming transactions**.
2. Contract accounts **can be set up as Multisig Accounts**.
3. A Multisig Account can be structured such that it has a daily limit that you specify, and **only if the daily limit is exceeded will multiple signatures be required**.

Disadvantages of CA:

1. Creating contract accounts costs gas because they use the valuable computational and storage resource of the network.
2. Contract accounts can't initiate new transactions on their own. Instead, contract accounts can only fire transactions in response to other transactions they have received either from an externally owned account or from another contract account.



Components of Ethereum Network - (5) Ethereum Account (EOA Vs CA)

Externally Owned Accounts	Contract Account
Controlled by third party	Controlled by Contract Code
Private Key is needed to access EOAs	No key is needed to access Contract Accounts
EOAs are created automatically on creating wallet	CA require EOAs to be activated
EOA doesn't have a code associated with it	CA have their own associated code
No execution fee is associated with EOAs	Execution fee is associated with CAs
Code hash = empty string	Code hash represents the code associated with the account



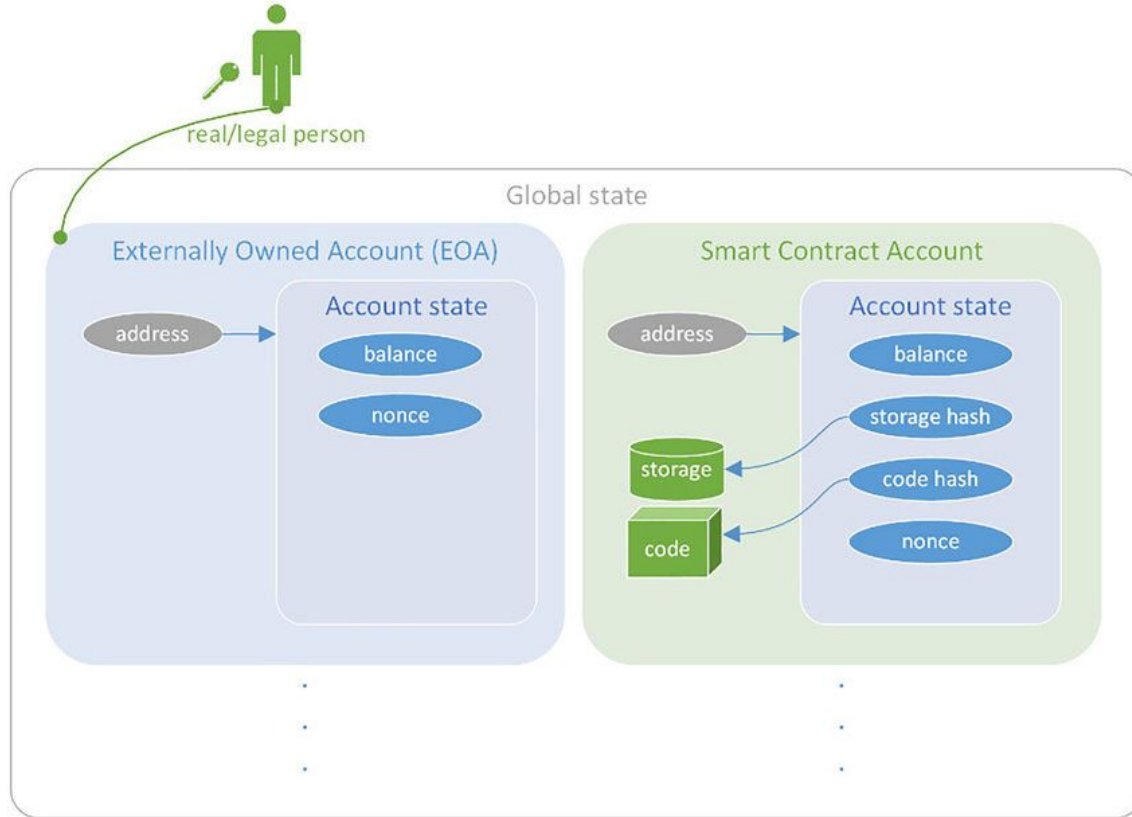
Components of Ethereum Network - (5) Ethereum Account (Field)

Different Fields in Ethereum Accounts

1. **Nonce:**
 - The nonce in an Ethereum account indicated the number of transactions that have been sent from that account.
 - This ensures that each transaction is made only once by taking count every time it takes place.
2. **Ether Balance:** the amount of ether present in an ether repository of the current ether account.
3. **Contract Code:**
 - This is non-mandatory to fill, in case it is present since not all accounts have a contract code.
 - But note, that they cannot be altered once executed.
4. **Storage:** This field remains not filled unless mentioned.
5. **Code Hash:**
 - hash that refers to the code present in that Ethereum account
 - since no code is associated with externally owned Ethereum accounts, ⇒ **code hash = empty string.**



Components of Ethereum Network - (5) Ethereum Account (Field)



Components of Ethereum Network - (5) Ethereum Accounts & Key Pairs

An Ethereum account is a **private-public key pair** that may be **linked to a blockchain address**.

Private Key

- It is a “owned” or “externally owned” account if the **private key is known and controlled by someone**.
- **Contract accounts** do not have a private key connected with them, although EOAs have.
- Control and access to one’s assets and contracts are granted through the EOA private key.
- The **user keeps the private key safe.**

Public Key

- Account’s **public key is open**.
- This key serves as the account’s identity.
- A one-way cryptographic function is **used to produce the public key from the private key.**

For example, if you create an account on Ethereum,

- **Retain the private key**
- **Share the public key.** As transactions between accounts are completed using public keys.



Components of Ethereum Network - Nonce, Storage Root, Ethash

6. Nonce

- **For EOAs**, nonce means the number of transactions via this account.
- **For CA**, nonce means the number of contracts generated via this account.

7. Storage Root

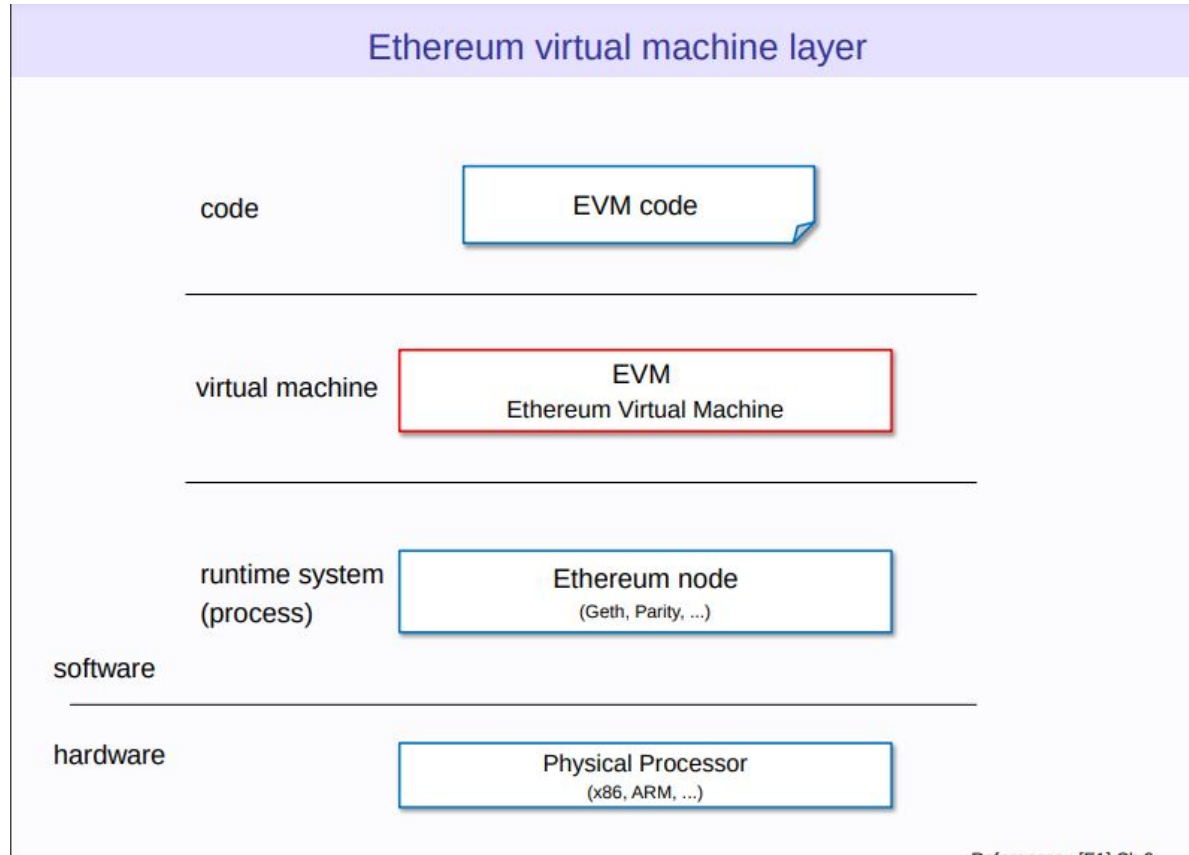
- It is the **main root node of a Merkle tree**.
- **Hash of all details of the account** is stored here.
- The **root of the Merkle tree** is the verification of all transactions.

8. Ethash

- **PoW algorithm for Ethereum 1.0**
- most **recent version of Dagger-Hashimoto**



Components of Ethereum Network - EVM

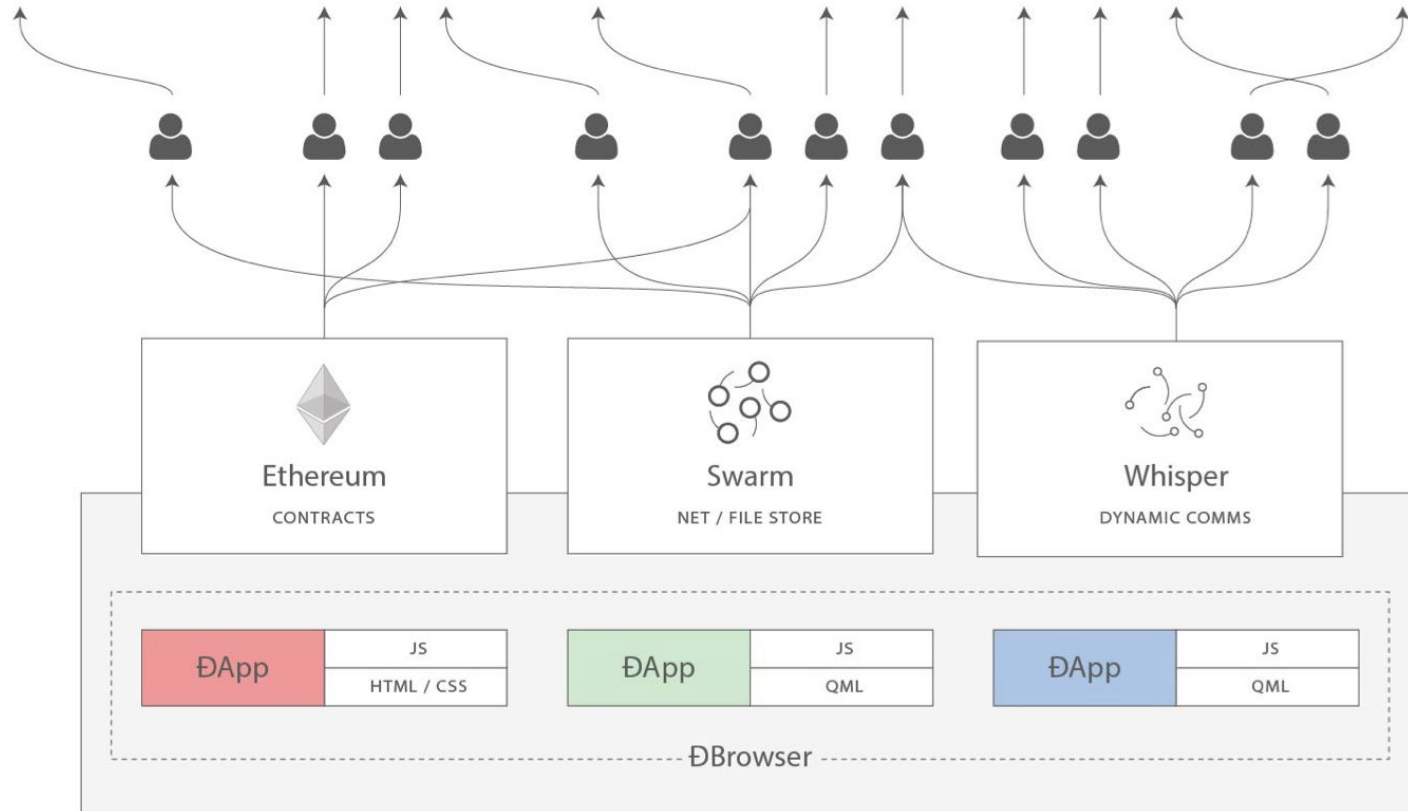


Components of Ethereum Network - Swarm & Whisper

- Swarm and Whisper are complementary technologies contributing to the vision of Ethereum as a "world computer".
- When imagining Ethereum as a metaphor for a shared computer, it should be noted that computation alone is not enough.
- For a computer to be fully useful, it also needs storage to "remember" things and bandwidth to "communicate" them. This could be summarised as such:
 - **Contracts:** decentralized logic
 - **Swarm:** decentralized storage
 - **Whisper:** decentralized messaging



Components of Ethereum Network - Swarm & Whisper



Interaction including Ethereum contracts, Swarm storage, Whisper comms



Components of Ethereum Network - Swarm & Whisper

Swarm

- designed as an accounting protocol that benefits from the automatic execution of so-called "smart contracts" running on the Ethereum Virtual Machine (EVM).
- This accounting protocol is independent of the physical storage mechanism.
- That is, it is **not intrinsically tied to a specific storage system**.
 - It could be [IPFS](#), [BitTorrent](#), or some future technology not yet invented.
- it is part of the [vision of a fully decentralized web](#).

Courtesy : [Ethereum Stackexchange](#)
[Oreilly - Whisper](#)
[Ethereum Blog](#)



Components of Ethereum Network - Swarm & Whisper

Whisper

- provides **decentralized peer-to-peer messaging capabilities** to the Ethereum network.
- It is an **identity based messaging system**
- It is a **communication protocol that DApps use to communicate with each other.**
- The data and routing of messages are **encrypted within Whisper communications.**
- uses the DEVP2p wire protocol for exchanging messages between nodes on the network.
- designed to be **used for smaller data transfers** and in scenarios where **real-time communication is not required.**
- **designed to provide a communication layer that cannot be traced**
- provides dark communication between parties.

Note : Blockchain can be used for communication, but that is expensive, and a consensus is not really required for messages exchanged between nodes.

- used as a **protocol that allows censor-resistant communication.**

Courtesy : [Ethereum Stackexchange](#)
[Oreilly - Whisper](#)
[Ethereum Blog](#)



Whisper

- At a considerable cost of bandwidth and latency, whisper $\Rightarrow$ deliver a 100% dark operation.
- that is **zero leakage of metadata during peer-to-peer communication**
- Normal communication protocol's $\Rightarrow$ to maximize the bandwidth and minimize latency.
- Goal: **to nullify leakage of metadata and achieve true darkness, where no third party can eavesdrop while two peers are communicating.**
 - For this, whisper is willing to give up on both bandwidth and latency constraints.
- Whisper messages are ephemeral (short lived) and have an associated time to live (TTL)



Components of Ethereum Network - Swarm & Whisper

Whisper

- Allows nodes in the network communicate with each other.
- Supports broadcasting, user-to-user, encrypted messages, and so on.
- It's **not designed to transfer bulk data.**
- **deliver secure messaging between peers without writing any information to the blockchain.**
- It was **part of the DevP2P wire protocol** but is **now deprecated.**

Courtesy : [Ethereum Stackexchange](#)
[Oreilly - Whisper](#)
[Ethereum Blog](#)



How Does Ethereum Work?

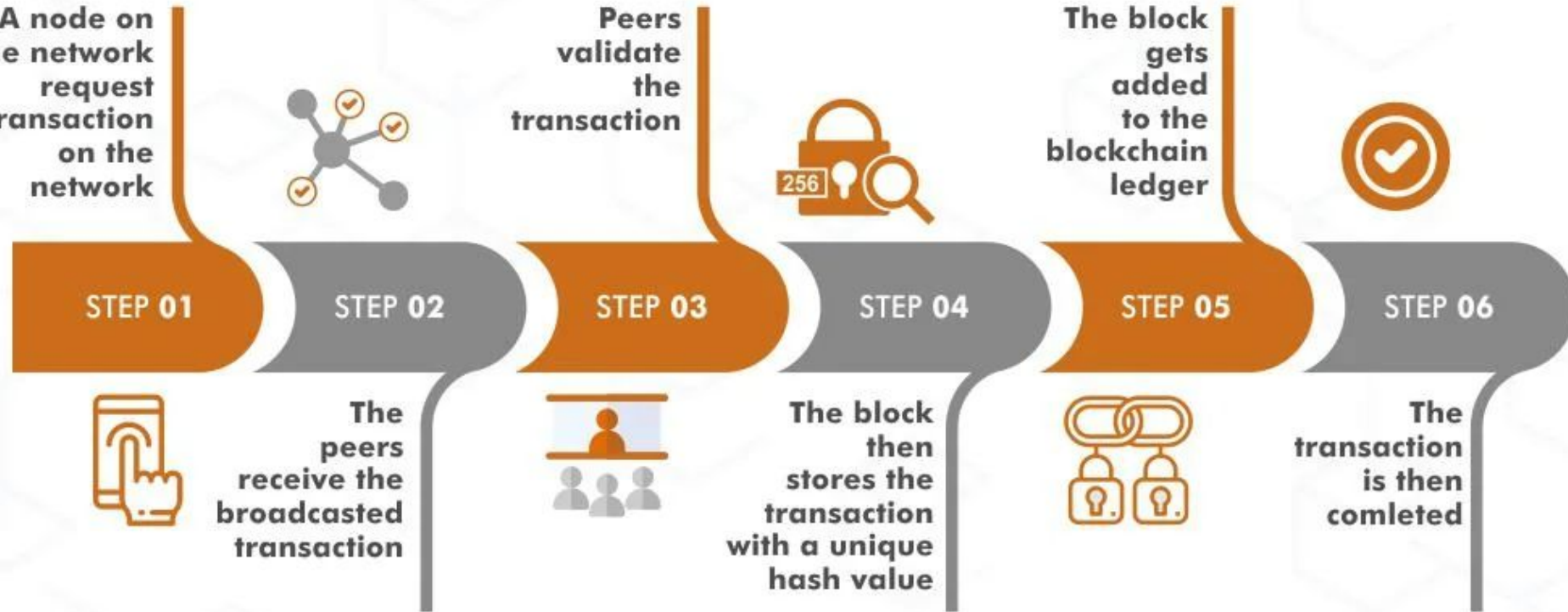
Ethereum implements an execution environment called **Ethereum Virtual Machine (EVM)**.

1. When a **transaction triggers a smart contract** all the nodes of the network will execute every instruction.
2. **All the nodes** will run **EVM** for **block verification**, where the nodes will go through the transactions listed in the block and runs the code as triggered by the transaction in the EVM.
3. **All the nodes** on the network must **perform the same calculations for keeping their ledgers in sync**.
4. Every transaction must include:
 - a. **Gas limit**.
 - b. **Transaction Fee** that the sender is willing to pay for the transaction.
5. If **total amount of gas needed to process the transaction $\leq$ the gas limit** then the transaction will be processed
6. if the **total amount of the gas $>$ the gas limit** then the transaction will not be processed and the fees are still lost.

Thus it is **safe to send transactions with the gas limit above the estimate to increase the chances of getting it processed**



Transactions in Blockchain - Life Cycle

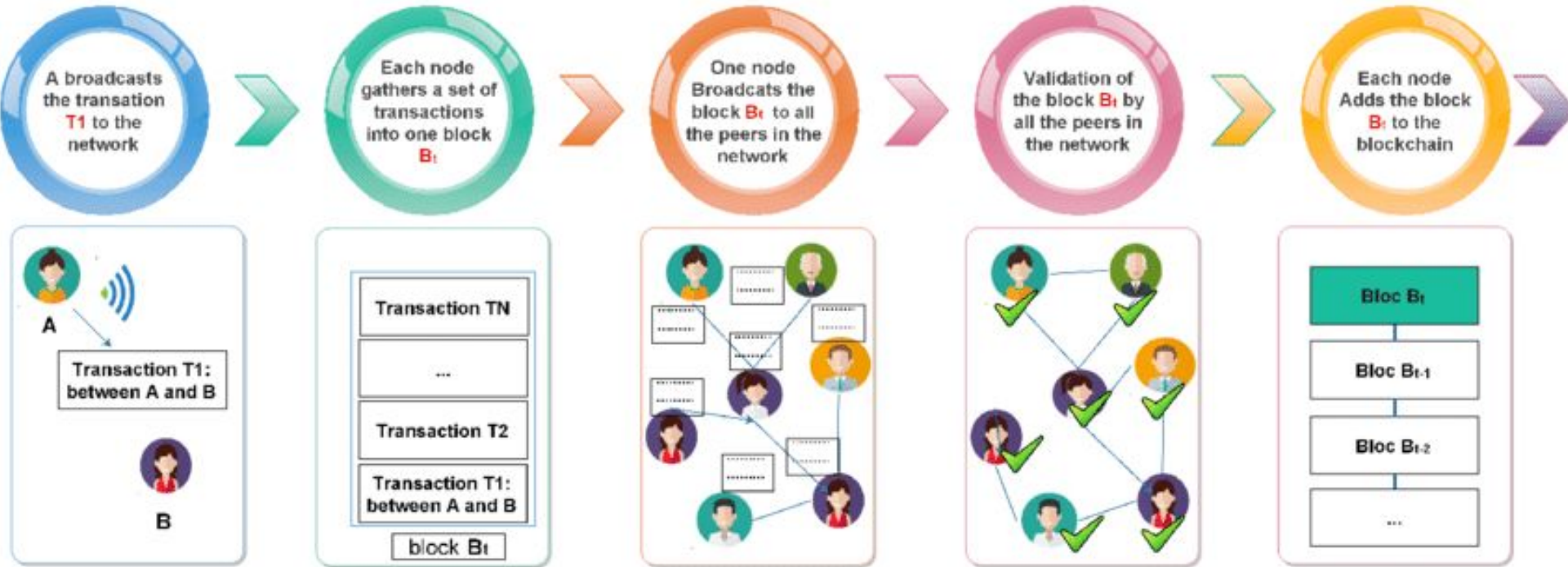


Transactions in Blockchain - Life Cycle

1. Someone requests a transaction. The transaction could involve cryptocurrency, contracts, records, or other information.
2. **Transaction is broadcast to all P2P** participation computers in the specific blockchain network. These are called **Nodes**. All transactions are published to the **Mempool** or memory pool, where they are considered 'pending'. **Gas fees** are paid by users as part of the transaction to compensate for the computing energy required to process and validate transactions on the blockchain.
3. **Miners** verify the transaction. Every computer in the network checks the transaction against some validation rules that are set by the creators of the specific blockchain network.
4. **Validated transactions** are stored into a block and are sealed with a lock referred to as the **Hash**.
5. **New block is added to the existing Blockchain**. This block becomes part of the blockchain when other computers in the network validate if the lock on the block is correct.
6. The transaction is complete. Now the transaction is part of the blockchain and cannot be altered in any way.



Transactions in Blockchain - Example



Transactions in Blockchain - Example

- Alice wants to send two coins to Bob.
 - Each transaction has **three main parts**:
 - **The input**: Alice's private coin address, she wants to spend.
 - **The output**: Bob's public key or coin address.
 - **Amounts**: the amount of coins Alice wants to spend.
1. **Alice signs a message with the transaction details using her private key.** The message contains the input, output, and amount to be sent.
 2. The **transaction is then broadcast to the network** saying the amount of coins in her account should go down by two. The amount in Bob's account should increase by two.
 3. **Each computer in the network will receive the message** and apply the requested transaction to its copy of the ledger, updating the account balances.
 4. **Add the transactions into the MemPool.**
 5. **Miner** select few transactions from MemPool and tries to **solve Cryptographic Puzzle**.
 6. On solving the Puzzle, the **Block is broadcasted to the network for validation**.
 7. After adding Block into the Blockchain, **Transactions added in the Block are later removed from MemPool**



Benefits of Ethereum

1. **Availability:**
 - As the Ethereum network is decentralized so **there is no downtime**.
 - **Even if one node goes down other computing nodes are available**.
2. **Privacy:** **Users don't need to enter their personal credentials** while using the network for exchanges, thus **allowing them to remain anonymous**.
3. **Security:** Ethereum is **designed to be unhackable**, as the **hackers have to get control of the majority of the network nodes to exploit the network**.
4. **Less ambiguity:** The **smart contracts that are used as a basis for trade and agreement** on Ethereum **ensure stronger contracts** than traditional contracts which require follow-through and interpretation.
5. **Rapid deployment:** On Ethereum decentralized networks, enterprises can easily deploy and manage private blockchain networks instead of coding blockchain implementation from scratch.
6. **Network size:** Ethereum network **can work with hundreds of nodes and millions of users**.
7. **Data coordination:** Ethereum **decentralized architecture better allocates information** so that the **network participants don't have to rely on a central entity** to manage the system and mediate transactions.



The DeFi Ecosystem - Smart Contracts

- A **self-executing program** that automates the actions required in an agreement or contract. Once completed, the transactions are trackable and irreversible.
- Smart contracts **permit trusted transactions and agreements to be carried out among disparate, anonymous parties** without the need for a central authority, legal system, or external enforcement mechanism.
- **do not contain legal language**, terms, or agreements
- **Only code that executes actions when specified conditions** are met.
- Nick Szabo, an American computer scientist
 - invented a virtual currency called "**Bit Gold**" in 1998,
 - defined smart contracts as computerized transaction protocols that execute the terms of a contract

The DeFi Ecosystem - Smart Contracts : Features

1. **Distributed:** Everyone on the network is guaranteed to have a copy of all the conditions of the smart contract and they cannot be changed by one of the parties. A smart contract is replicated and distributed by all the nodes connected to the network.
2. **Deterministic:** Smart contracts can only perform functions for which they are designed only when the required conditions are met. The final outcome will not vary, no matter who executes the smart contract.
3. **Immutable:** Once deployed smart contract cannot be changed, it can only be removed as long as the functionality is implemented previously.
4. **Autonomy:** There is no third party involved. The contract is made by you and shared between the parties. No intermediaries are involved which minimizes bullying and grants full authority to the dealing parties. Also, the smart contract is maintained and executed by all the nodes on the network, thus removing all the controlling power from any one party's hand.
5. **Customizable:** Smart contracts have the ability for modification or we can say customization before being launched to do what the user wants it to do.
6. **Transparent:** Smart contracts are always stored on a public distributed ledger called blockchain due to which the code is visible to everyone, whether or not they are participants in the smart contract.
7. **Trustless:** These are not required by third parties to verify the integrity of the process or to check whether the required conditions are met.
8. **Self-verifying:** These are self-verifying due to automated possibilities.
9. **Self-enforcing:** These are self-enforcing when the conditions and rules are met at all stages.

The DeFi Ecosystem - Smart Contracts : Benefits



Autonomy



Accuracy



Transparency



High Speed



Data storage



Trustability



Cost Savings



Robust Backup

The DeFi Ecosystem - Smart Contracts : Types

1. Smart Legal Contracts

- Legally enforceable
- Require the parties to fulfill their contractual obligations.
- Failure to do so may result in strict legal actions against them.

2. Decentralized Autonomous Organizations (DAO)

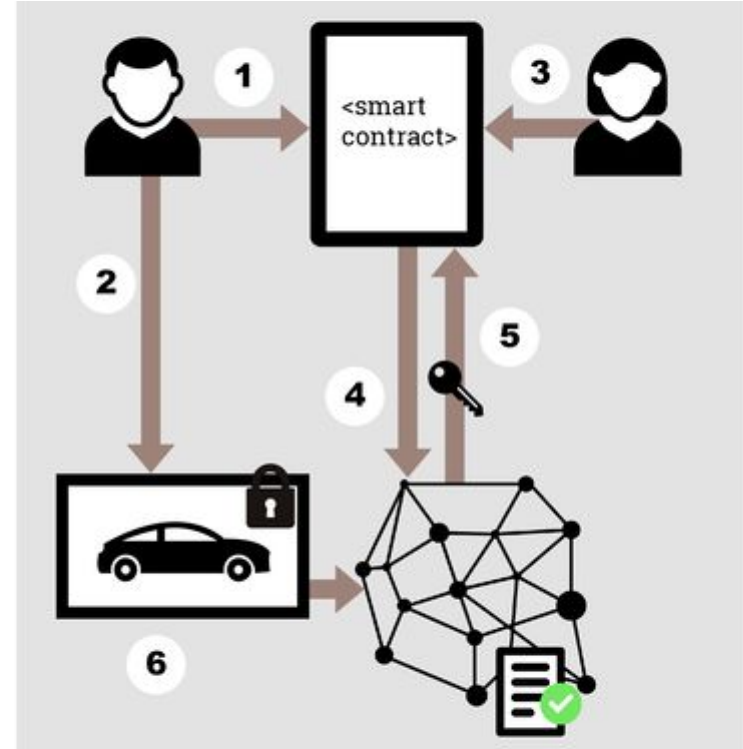
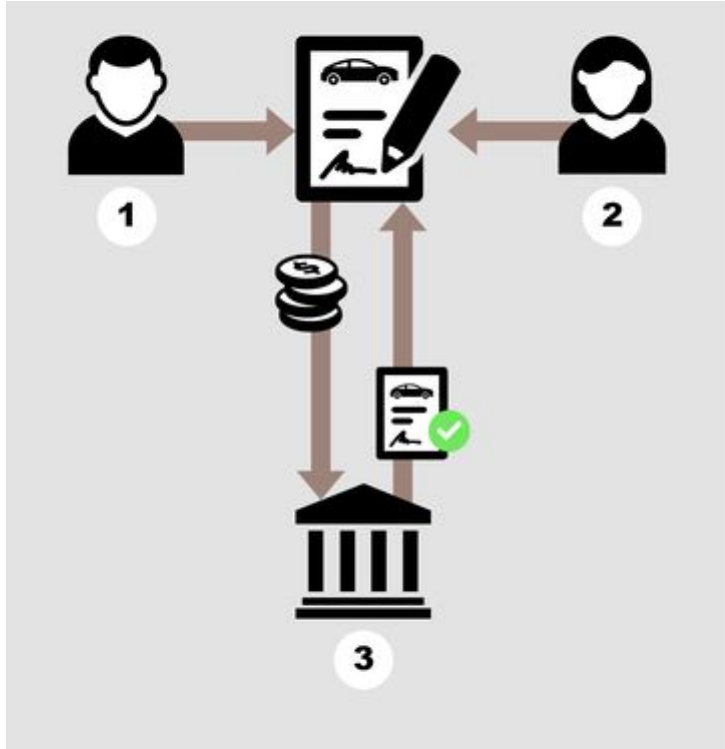
- These are blockchain communities that are bound to specific rules coded into blockchain contracts combined with governance mechanisms.
- Any action taken by the community members gets replaced by a self-enforcing code.

3. Application Logic Contracts (ALC)

- contain an application-based code that remains in sync with other blockchain contracts.
- It enables communication across different devices, such as the merger of the Internet of Things with blockchain technology.

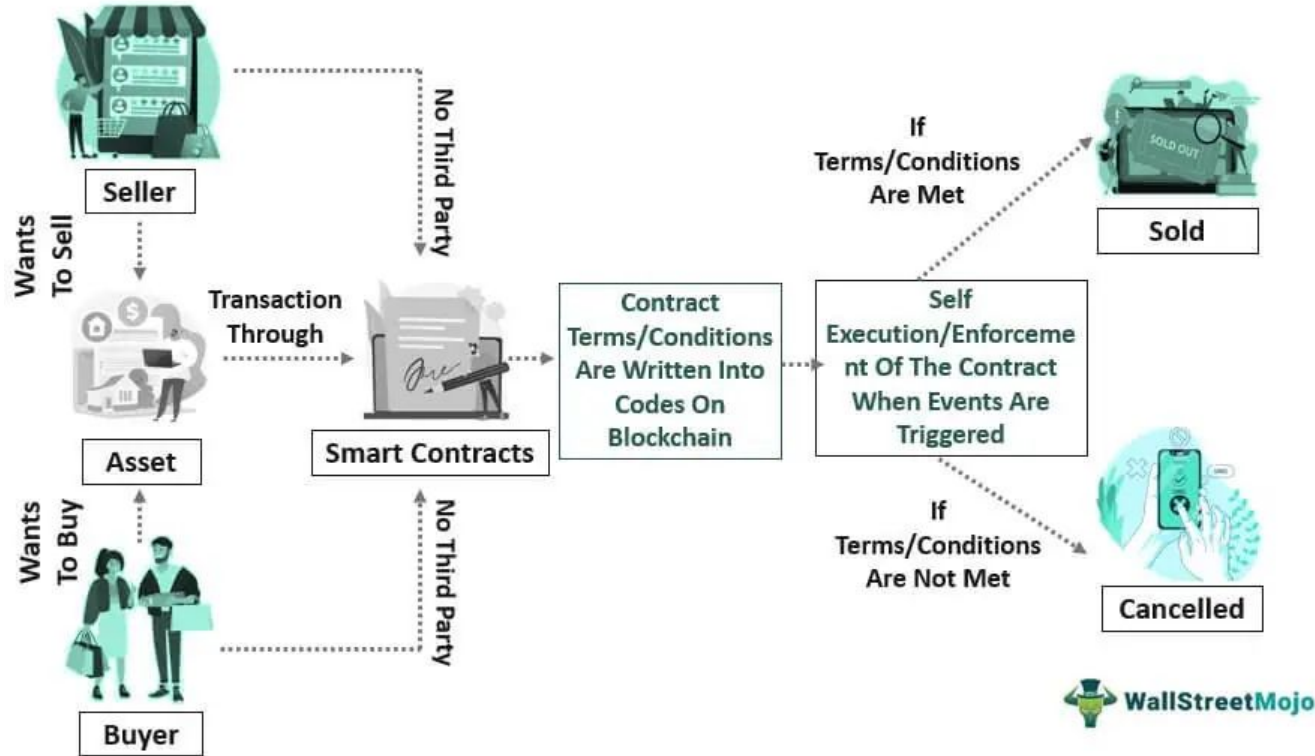
The DeFi Ecosystem - Smart Contracts

Traditional Contracts Vs Smart Contracts



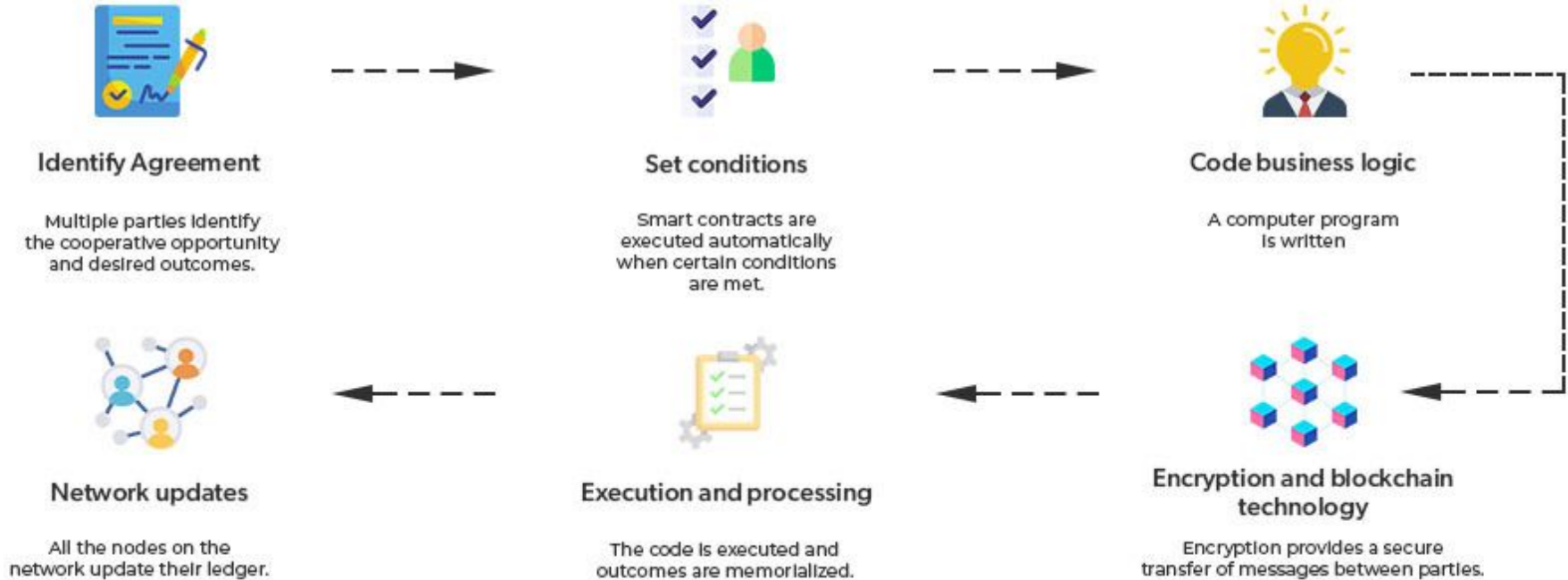
The DeFi Ecosystem - Smart Contracts

Smart Contract Functioning



The DeFi Ecosystem - Smart Contracts

How do smart contracts work ?

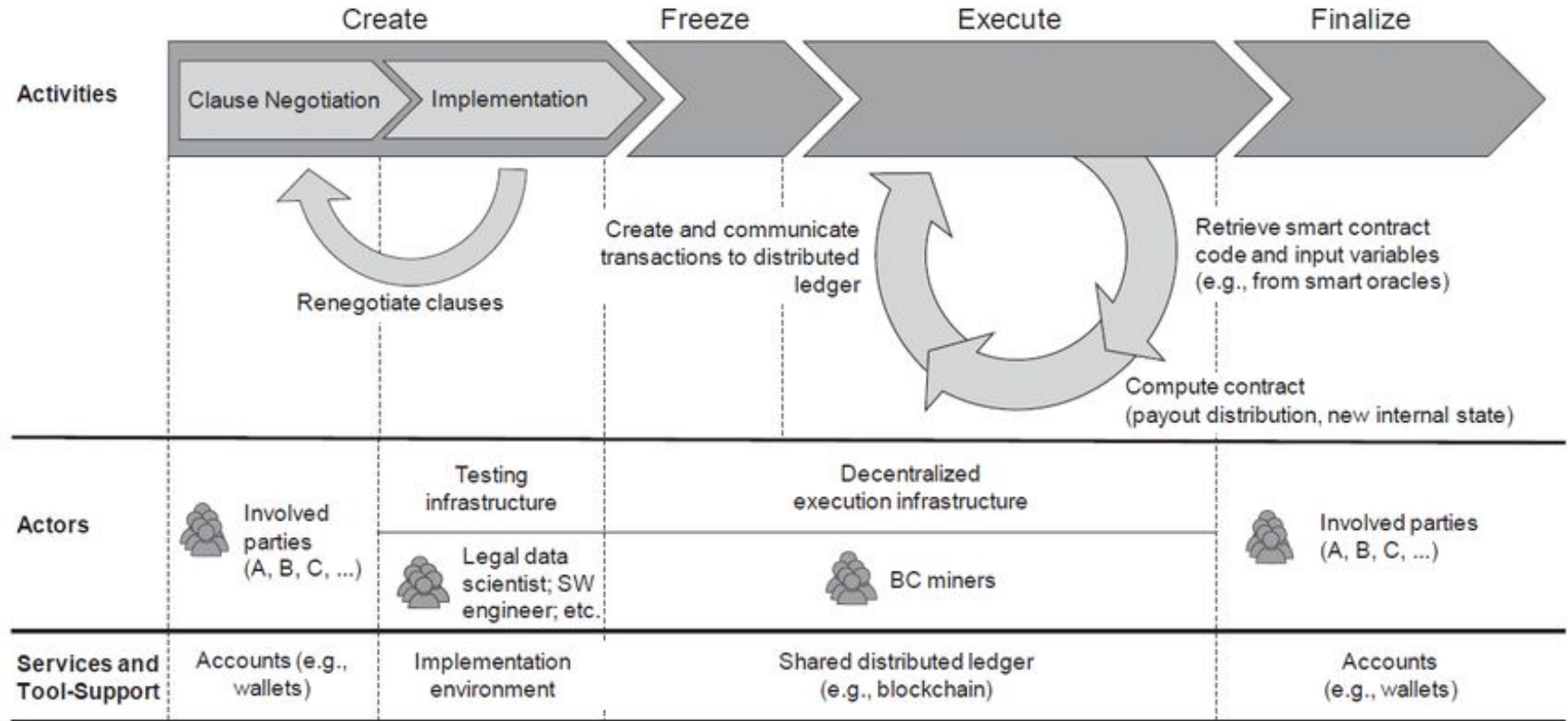


How do smart contracts work ?

1. **Identify Agreement:** Multiple parties identify the cooperative opportunity and desired outcomes and agreements could include business processes, asset swaps, etc.
2. **Set conditions:** Smart contracts could be initiated by parties themselves or when certain conditions are met like financial market indices, events like GPS locations, etc.
3. **Code business logic:** A computer program is written that will be executed automatically when the conditional parameters are met.
4. **Encryption and blockchain technology:** Encryption provides secure authentication and transfer of messages between parties relating to smart contracts.
5. **Execution and processing:** In blockchain iteration, whenever consensus is reached between the parties regarding authentication and verification then the code is executed and the outcomes are memorialized for compliance and verification.
6. **Network updates:** After smart contracts are executed, all the nodes on the network update their ledger to reflect the new state. Once the record is posted and verified on the blockchain network, it cannot be modified, it is in append mode only.



Life Cycle of a Smart Contract



Life Cycle of a Smart Contract

1. Creation Phase:

- consists of iterative contract negotiation and an implementation phase.
- First, the parties must agree on the broad content and goals of the contract.
- Similar to typical contract negotiations and can be conducted online or in person contracts
- During this phase, the following tasks are completed:
 1. Multiple-party bargaining.
 2. Design, implementation, and validation of smart

2. Freeze:

- The validation of transactions on a blockchain is performed by a network of computers known as nodes. The blockchain miners are these nodes.
- To prevent the ecosystem from being swamped with smart contracts, miners must be paid a tiny fee in exchange for this service.



Life Cycle of a Smart Contract

3. Execution:

- Contracts placed on the distributed ledger are read by participating nodes.
- The authentication nodes validate the integrity of a smart contract,
- the code is performed by the smart contract interference engine (or by the compiler).
- When one party's inputs for execution are received in the form of coins (commitment to goods via coins), the interference engine **generates a transaction triggered by the met criterion.**
- The execution of the smart contract **results in a new set of transactions and a new state for the smart contract.**
- The **discoveries and new state information** are entered into the distributed ledger and validated using the consensus procedure.

4. Finalize:

- the resulting transactions and updated state information are recorded in the distributed ledger and confirmed via the consensus process.
- The previously pledged digital assets are transferred (assets are unfrozen), and the contract is signed to confirm all transactions.



2. The DeFi Ecosystem - DApps

- A software application that operates on a decentralized network, typically using blockchain technology.
- Compared to traditional applications that rely on centralized servers, Dapps leverage the security, transparency, and trustlessness of blockchain to function.
- Salient Features of DApps
 1. **Decentralization:**
 - Dapps run on a decentralized network of computers, often referred to as a blockchain.
 - This network is distributed across many nodes, making it resistant to single points of failure and censorship.
 2. **Smart Contracts:**
 - Dapps typically use smart contracts, which are self-executing contracts with the terms of the agreement directly written into code.
 - Smart contracts automatically execute actions when specific conditions are met, without the need for intermediaries.
 3. **Transparency:**
 - All transactions and actions within a Dapp are recorded on a public ledger (the blockchain).
 - This transparency ensures that anyone can verify transactions and data, enhancing trust in the application.



Introduction to Dapp (Decentralized Application)

4. Security:

- Blockchain technology, with its cryptographic techniques, provides a high level of security.
- Transactions are immutable, meaning once recorded on the blockchain, they cannot be altered.
- This makes Dapps resilient to fraud and hacking.

5. Trustlessness:

- Dapps aim to operate without relying on a central authority or intermediary.
- Users can interact with the application and each other directly without needing to trust a third party.

6. Token Economy:

- Many Dapps have their own native tokens or use established cryptocurrencies like Ethereum's Ether.
- These tokens can be used for various purposes within the application, such as paying for services, participating in governance, or earning rewards.

7. Use Cases:

- Dapps have a wide range of use cases, including decentralized finance (DeFi), non-fungible tokens (NFTs), supply chain management, voting systems, gaming, and more.
- Each Dapp is designed to solve specific problems or provide new functionalities in a decentralized manner.



Introduction to Dapp (Decentralized Application)

8. User Interface:

- Dapps often have user interfaces (UIs) similar to traditional apps or websites, making them accessible to a broader user base.
- Users may not even be aware that they are interacting with blockchain technology.

9. Challenges:

- While Dapps offer numerous advantages, they also face challenges, including scalability issues, user adoption barriers, and regulatory compliance.
- These challenges are actively being addressed by the blockchain community.

10. Development:

- Building a Dapp typically involves programming smart contracts using languages like Solidity (for Ethereum) and developing a frontend interface.
- Popular Dapp development frameworks like Truffle and web3.js make the development process more accessible.



Dapp Architecture

1. Ethereum blockchain:

- Securely executes and verifies application code, called smart contracts.
- DApps use the Ethereum blockchain for data storage.

2. Smart Contracts:

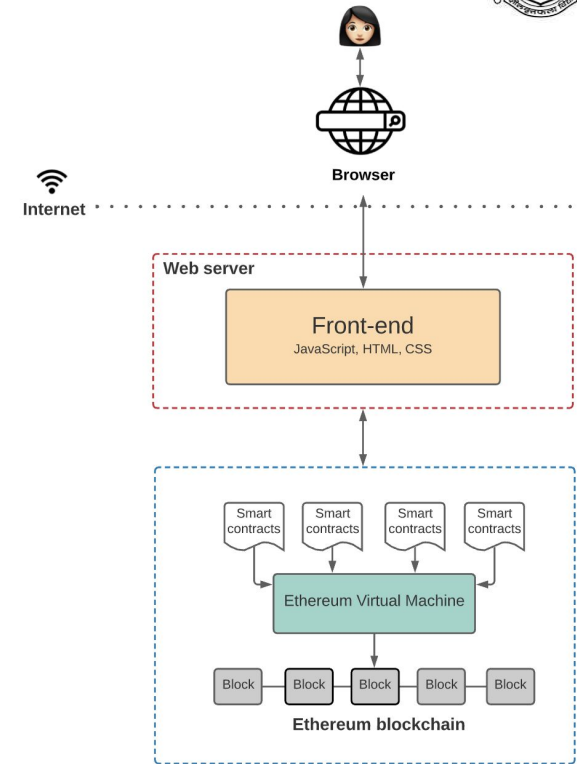
- DApps use smart contracts to define the state changes happening on the blockchain.
- A smart contract is a collection of code and data that resides at a specific address on the Ethereum Blockchain and runs on the Ethereum blockchain.

3. Ethereum Virtual Machine(EVM):

- Global virtual computer that executes the logic defined in the smart contracts and processes the state changes that happen on this Ethereum network.

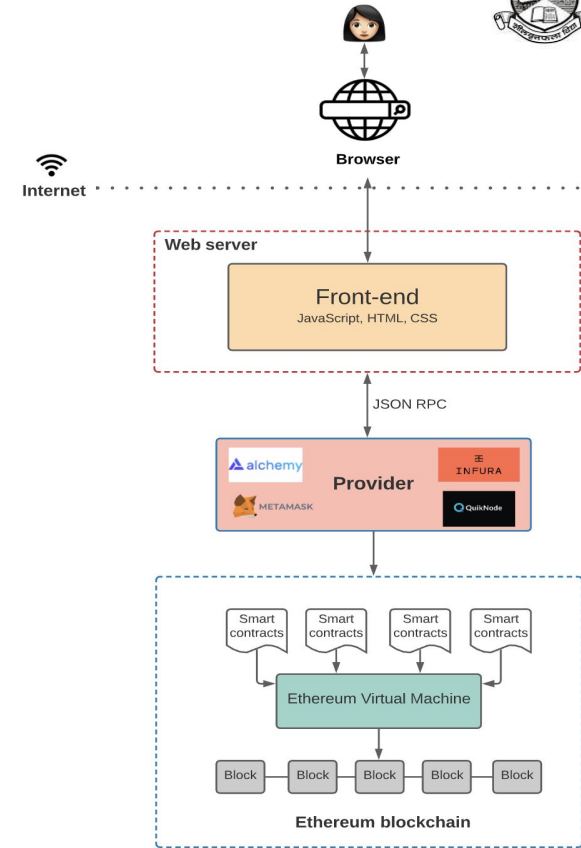
4. Front-end:

- The part of the DApps, that users can see and interact with such as the graphical user interface(GUI),
- Communicates with the application logic defined in smart contracts.



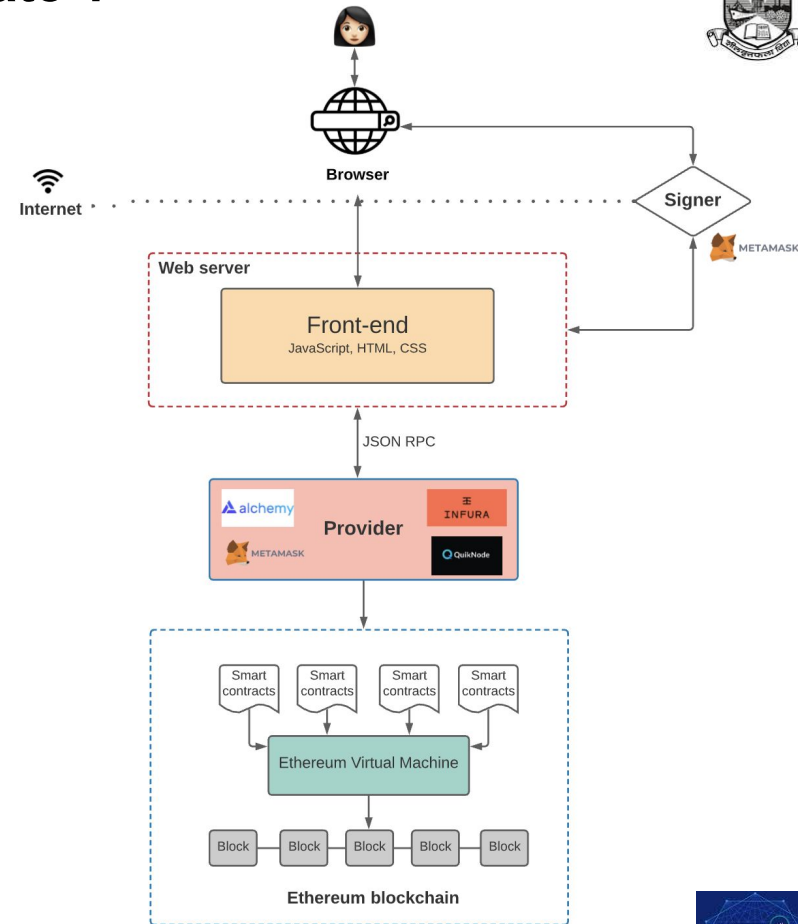
DApp - How Frontend Code Communicate ?

- When we need to interact with the data and code on a blockchain, we need to interact with one of these nodes.
- As any node can broadcast a request for a transaction to be executed on the EVM.
- A miner will then execute the transaction and propagate the resulting state change to the rest of the network.
- There are two ways to broadcast a new transaction:
 1. Set up your own node which runs the Ethereum blockchain software
 2. Use nodes provided by third-party services like [Infura](#), [Alchemy](#), and [Quicknode](#)
- Every Ethereum client (i.e. provider) implements a JSON-RPC specification. This ensures that there's a uniform set of methods when frontend applications want to interact with the blockchain.



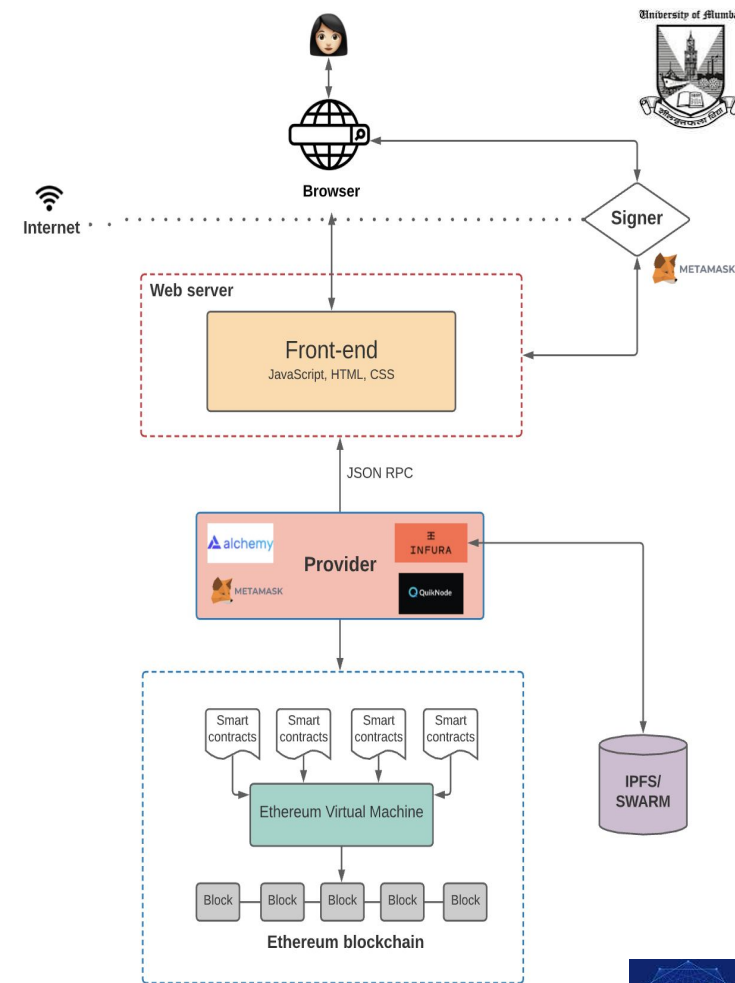
DApp - How Frontend Code Communicate ?

- When a user wants to publish a new post onto the chain, our DApp would ask the user to “sign” the transaction using their private key — only then would the DApp relay the transaction to the blockchain.
- Otherwise, the nodes wouldn't accept the transaction.
- This “signing” of transactions is where [Metamask](#) typically comes in.



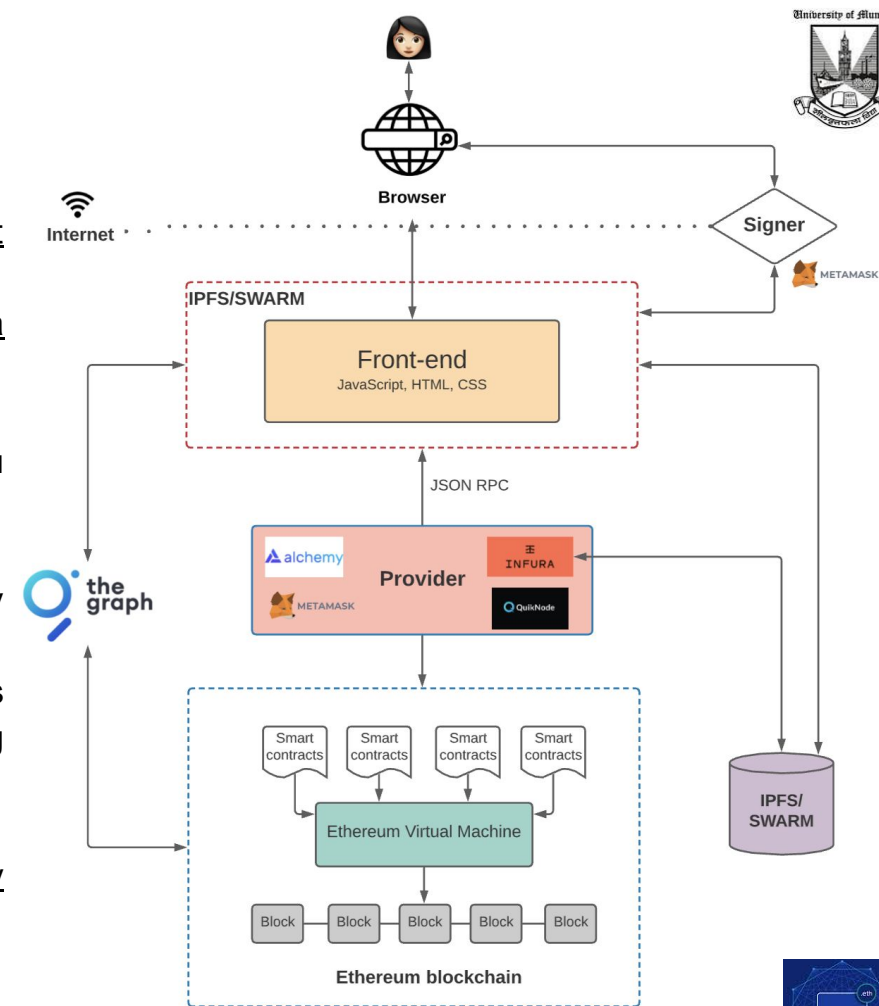
DApp - Storage on the Blockchain

- Storing everything on the blockchain gets really expensive.
- Users to pay extra for using your DApp every time their transaction requires adding a new state is not the best user experience.
- One way to combat this is to **use a decentralized off-chain storage solution**, like [IPFS](#) or [Swarm](#).
- IPFS : Distributed file system for storing and accessing data.
 - distributes and stores the data in a peer-to-peer network.
 - Has an incentive layer known as “Filecoin.” This layer incentivizes nodes around the world to store and retrieve this data.
 - IPFS providers
 - **Infura** (which provides you with an IPFS node) or
 - Pinata (“pin” your files to IPFS and take the IPFS hash and store that on the blockchain).
- **Swarm** : A decentralized storage network,
 - **Difference** : Swarm’s incentive system is built-in and enforced through smart contracts on the Ethereum blockchain for storing and retrieving data.



2. The Graph

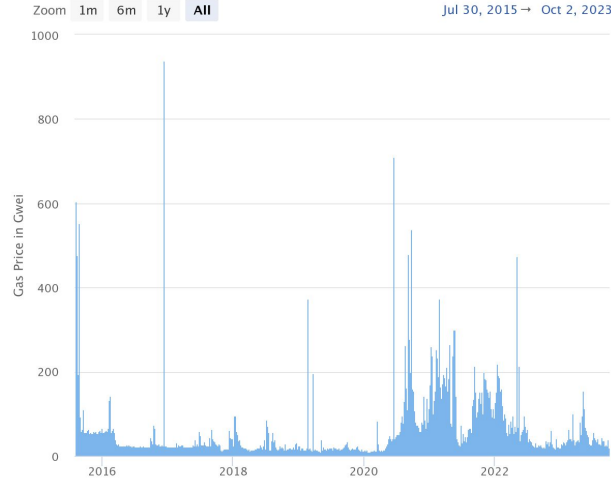
- Use the Web3.js library to query and listen for smart contract events.
 - Here, we need to listen to specific events and specify a callback every time the event is fired.
 - using callbacks to handle various UI logic gets very complex
 - Issue: when you deploy a smart contract and later realize you need an event emitted that you didn't originally include.
- ## 2. The Graph
- An off-chain indexing solution that makes it easier to query data on the Ethereum blockchain.
 - Allows to define which smart contracts to index, which events and function calls to listen to, and how to transform incoming events into entities that the frontend logic can consume.
 - It uses **GraphQL** as a query language,
 - By indexing blockchain data, The Graph lets us query on-chain data in our application logic with low latency



Ethereum Average Gas Price Chart

Source: Etherscan.io

Click and drag in the plot area to zoom in



Average Transaction Fee Chart

Source: Etherscan.io

Click and drag in the plot area to zoom in



Ethereum Network Transaction Fee Chart

Source: Etherscan.io

Click and drag in the plot area to zoom in



Note : Building a DApp on Ethereum with high gas fees and full blocks leads to a very bad UX.

Dapp Scalability

- to handle a growing user base and increasing transaction volume while maintaining a smooth and cost-effective user experience.
- Some strategies and approaches to address Dapp scalability:
 1. **Layer 2 Solutions:** : Protocols built on top of existing blockchains that enable faster and more scalable transactions. Examples include:
 - a. **Sidechains:** These are separate blockchains that can communicate with the main blockchain. They can handle transactions more quickly and with lower fees.
 - b. **State Channels:** State channels allow users to conduct off-chain transactions, reducing the burden on the main blockchain and improving speed and cost-effectiveness.
 - c. **Plasma:** Plasma chains are a framework for creating child chains that can process a high volume of transactions and periodically commit data to the main blockchain.
 2. **Sharding:** A technique where the blockchain network is divided into smaller, interconnected chains or "shards." Each shard can process its transactions independently, significantly increasing the network's throughput.
 3. **Optimistic Rollups:** A type of Layer 2 solution
 - a. Allows most transactions to be processed off-chain, with the main blockchain only used for dispute resolution.
 - b. Greatly reduces congestion and fees.



Dapp Scalability

4. **Blockchain Upgrades: through protocol upgrades.**
 - a. For example, Ethereum is transitioning from a proof-of-work (PoW) to a proof-of-stake (PoS) consensus mechanism with Ethereum 2.0, which is expected to increase transaction capacity.
5. **State Management:** is crucial for Dapp scalability.
 - a. Minimize on-chain storage and use techniques like Merkle trees to represent complex data structures more efficiently.
6. **Gas Optimization:**
 - a. Minimize the use of gas (transaction fees) on the blockchain by optimizing smart contract code and transaction execution. This can reduce costs for users and make the Dapp more accessible.
7. **Caching and Load Balancing:**
 - a. Implement caching mechanisms and load balancing to distribute traffic and reduce the load on individual nodes. This can improve response times and overall performance.
8. **Off-Chain Computation:**
 - a. Off-load computationally intensive tasks to off-chain servers or services, while still maintaining the security and trustlessness of critical operations on the blockchain.



Dapp Scalability

9. Token Design:

- a. Carefully design the tokenomics of your Dapp. Ensure that the token has utility within the Dapp and mechanisms for handling increased demand without causing congestion on the blockchain.

10. Governance and Community Engagement:

- a. Involve the Dapp's community and token holders in governance decisions related to scalability upgrades. This can lead to more consensus-driven and efficient scaling solutions.

11. Monitoring and Testing:

- a. Continuously monitor the Dapp's performance and scalability. Conduct stress tests to identify bottlenecks and areas for improvement.

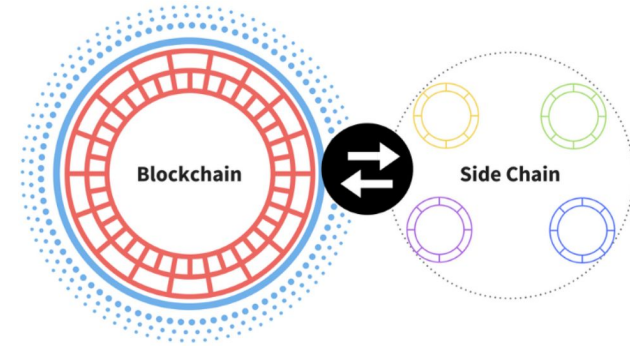
12. Hybrid Solutions:

- a. Consider a hybrid approach, combining on-chain and off-chain solutions to strike a balance between security and scalability.



Dapp Scalability

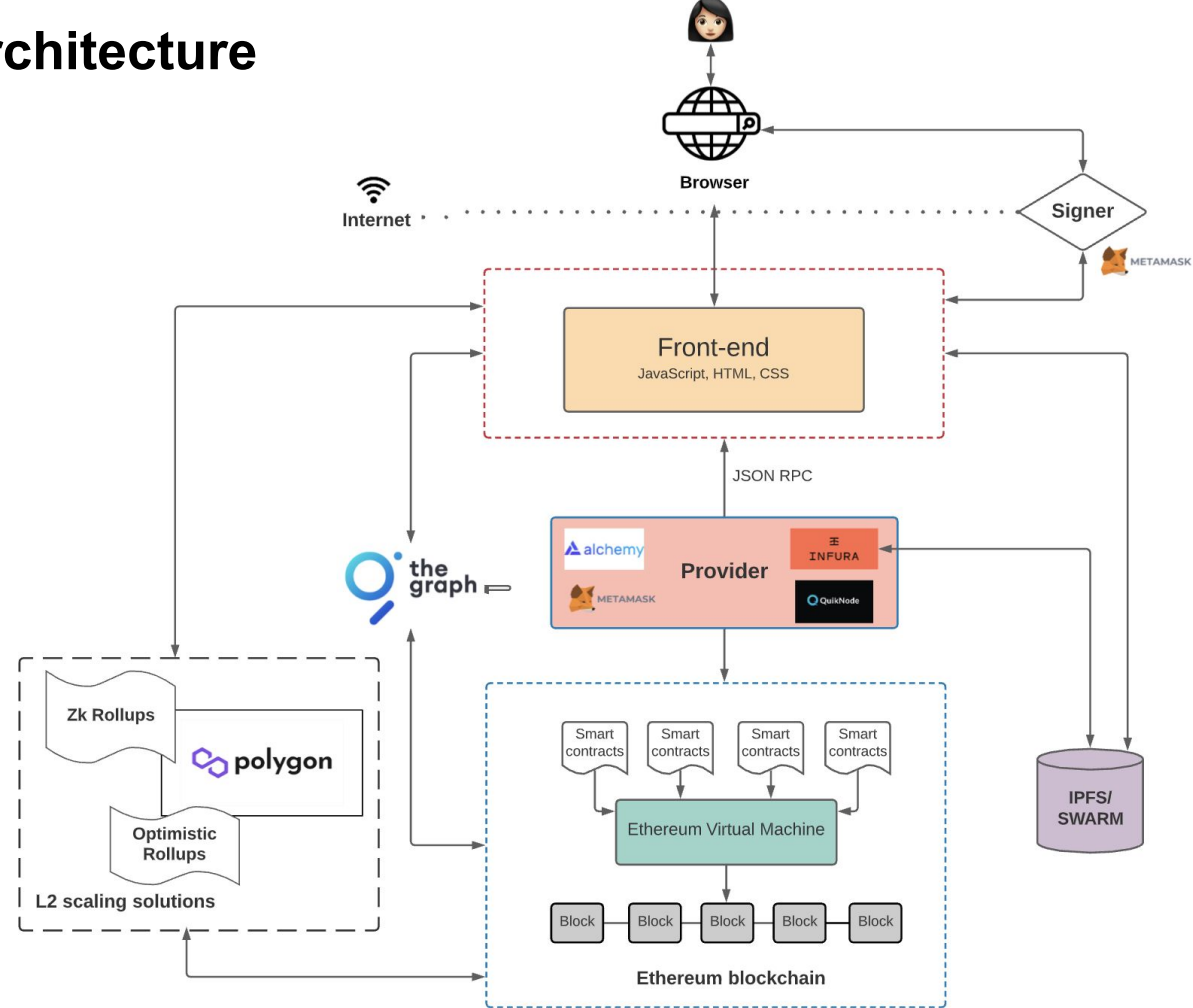
- L2 solutions do transaction execution off-chain, with only the transaction data stored on-chain.
- Helps us scale the blockchain because **we don't have to execute every single transaction on-chain.**
- This also **makes transactions faster and cheaper** — and they can still communicate with the main Ethereum blockchain when necessary.
- Popular Scalability Solution : [Polygon](#), an L2 scaling solution.
 - Polygon has “sidechains” that process and execute transactions.
 - A sidechain is a secondary blockchain that interfaces with the main chain. Every so often, the sidechain submits an aggregation of its recent blocks back to the primary chain.
- Other L2 solutions are [Optimistic Rollups and zkRollups](#).
 - Transactions are **batched off-chain using a “rollup” smart contract and then periodically commit these transactions to the main chain.**



Overall DApp Architecture

Addressing

- Storage,
- Querying
- Scalability



The DeFi Ecosystem - User & Liquidity Providers

DeFi Users

- Retail users
- Institutional users
- Developers & builders
- Arbitrageurs & traders

Role of DeFi Users

- Trading assets
- Lending & borrowing
- Yield farming
- Governance participation

DeFi Liquidity Providers

- Users who supply assets to liquidity pools
- Enable trading without order books
- Earn rewards and fees

Role of Liquidity Providers (LPs)

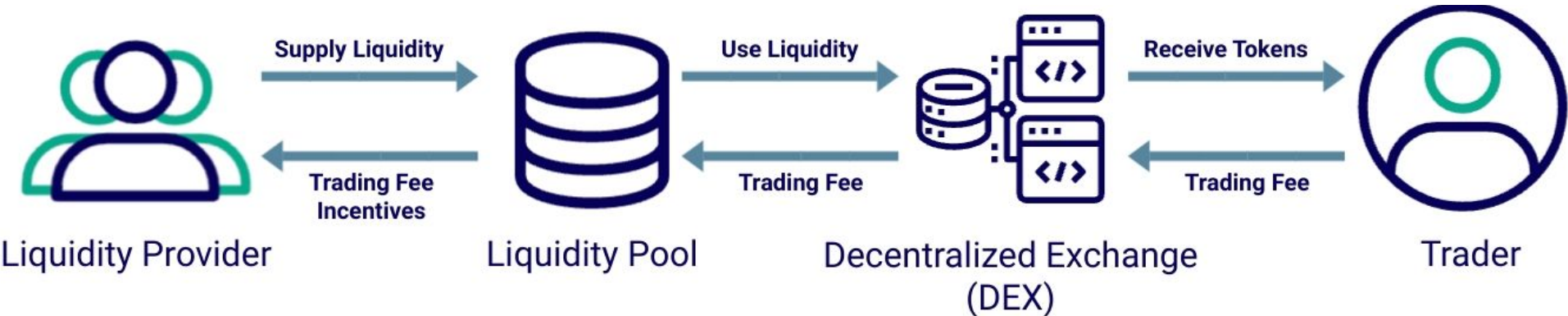
- Enables smooth trading
- Reduces price slippage
- Ensures protocol stability

The DeFi Ecosystem - How Liquidity Pools Work?

- Pair of assets locked in smart contracts
- Automated Market Maker (AMM) model
- Prices set algorithmically

User – Liquidity Provider Flow

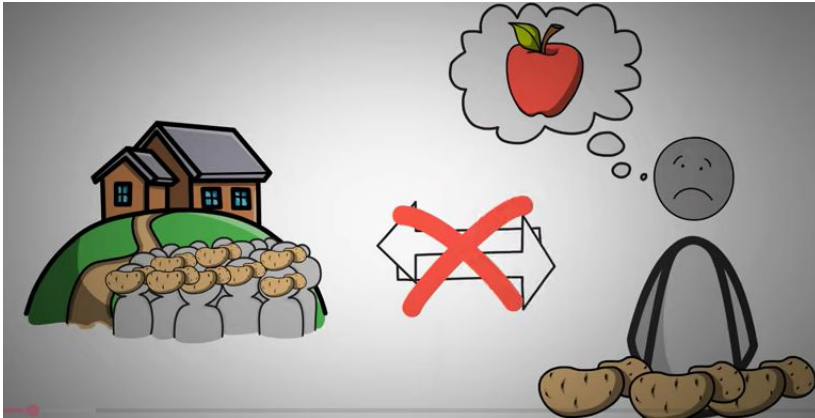
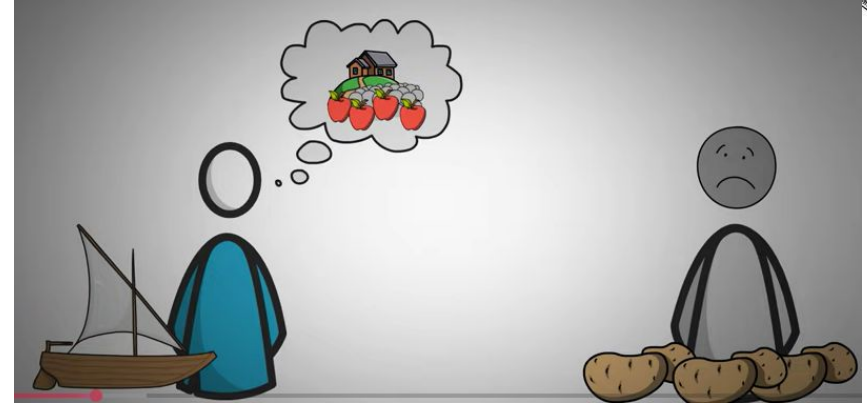
1. LP deposits Token A & B
2. Pool enables swaps
3. User pays fee
4. LP earns fee share



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [Techsling](#)

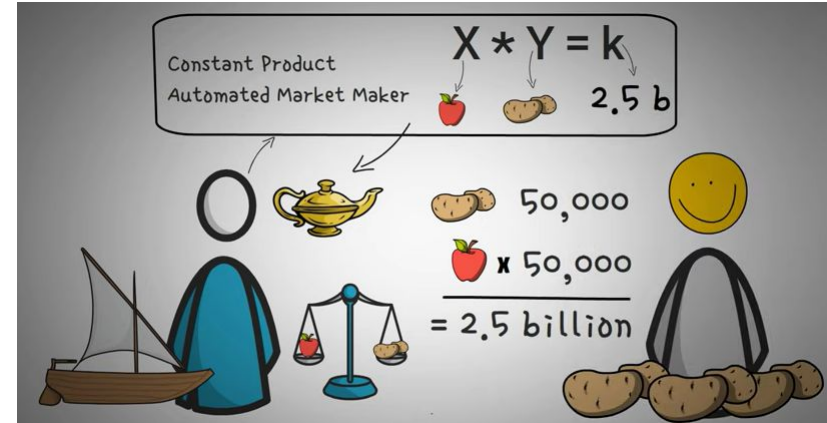
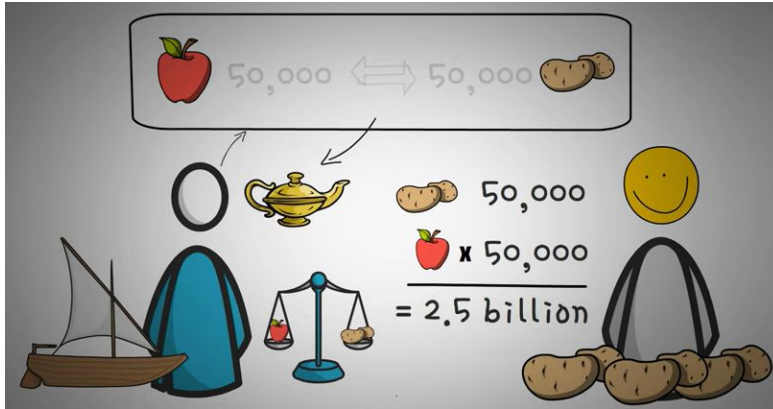
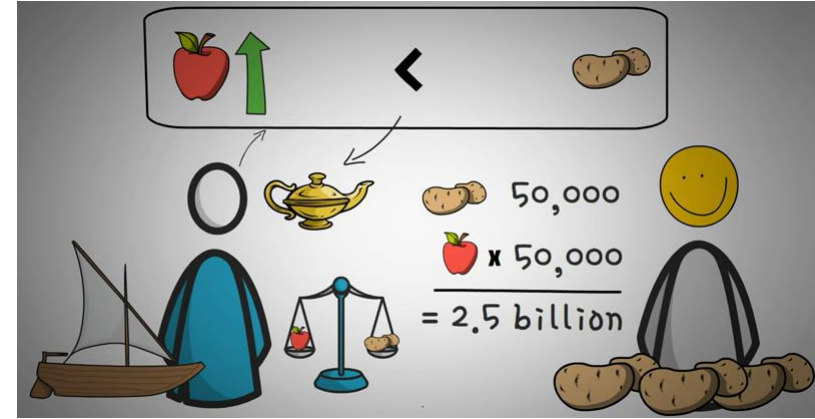
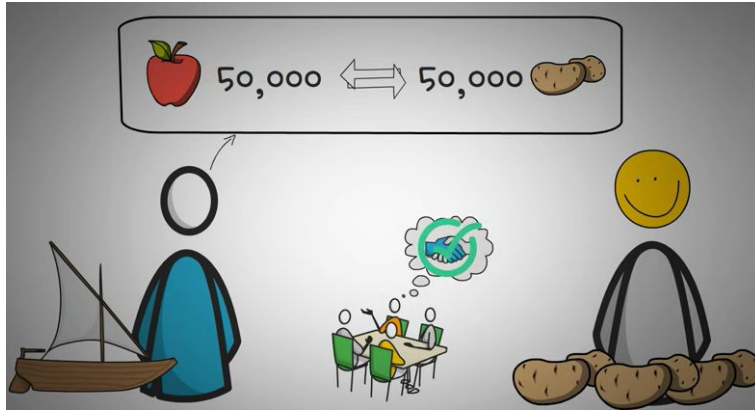
The DeFi Ecosystem - AMM Liquidity Pool



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [YouTube video : AMM Model](#)

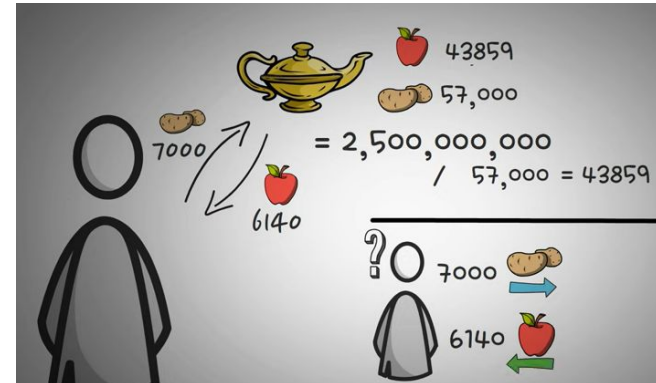
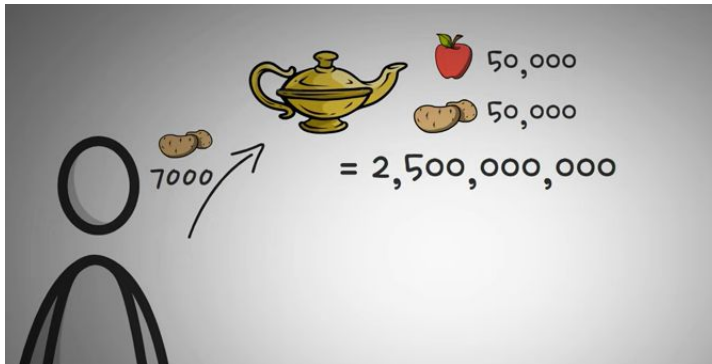
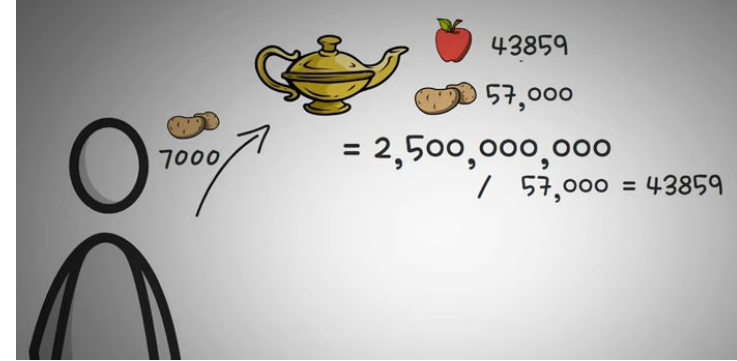
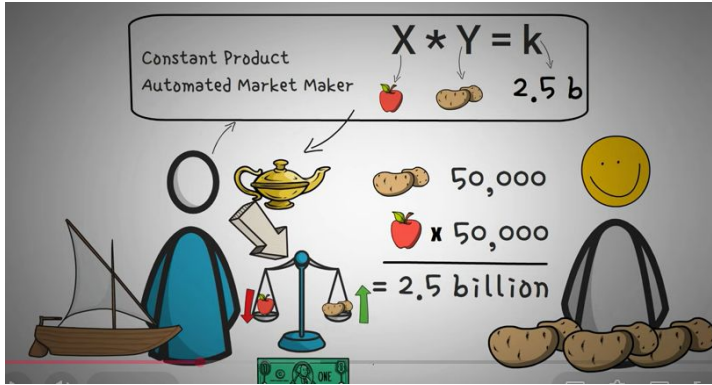
The DeFi Ecosystem - AMM Liquidity Pool



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [YouTube video : AMM Model](#)

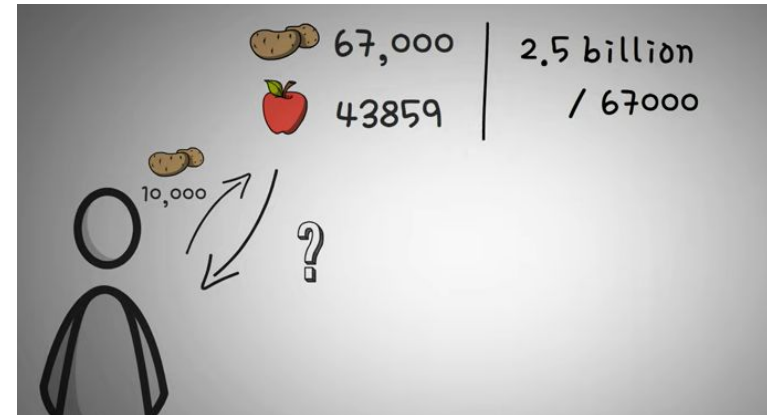
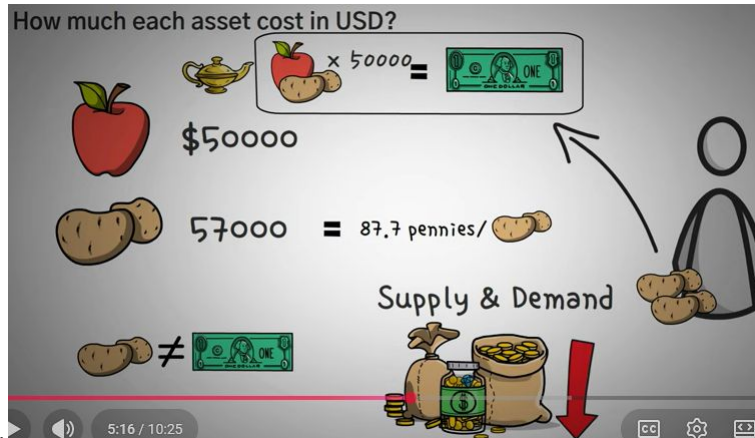
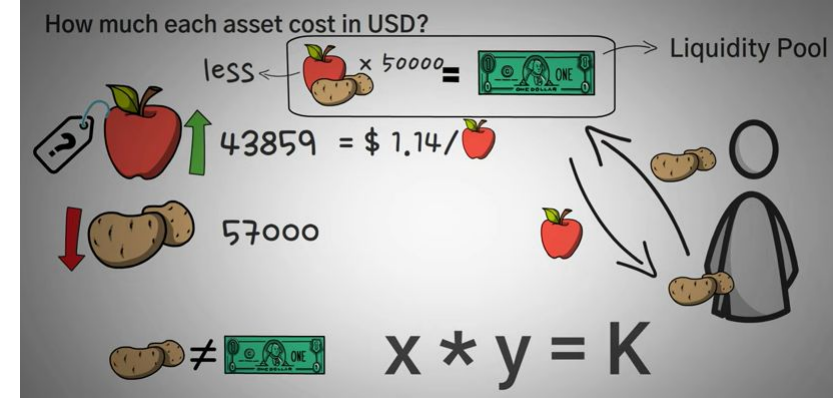
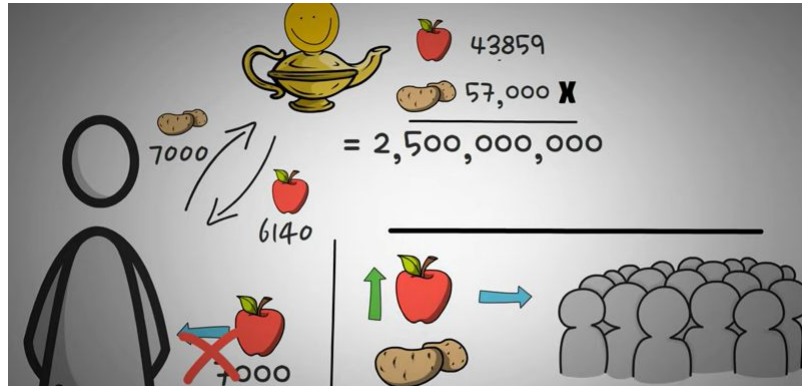
The DeFi Ecosystem - AMM Liquidity Pool



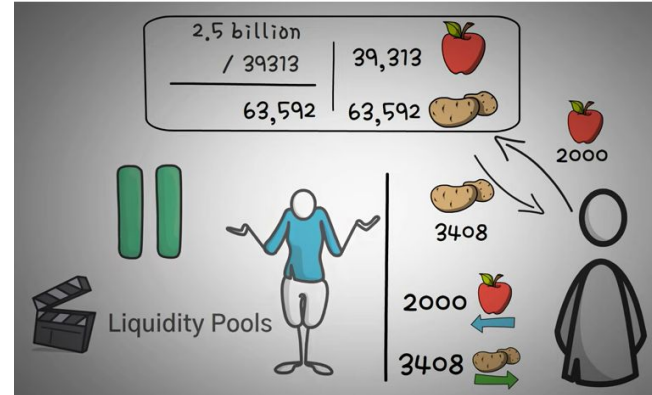
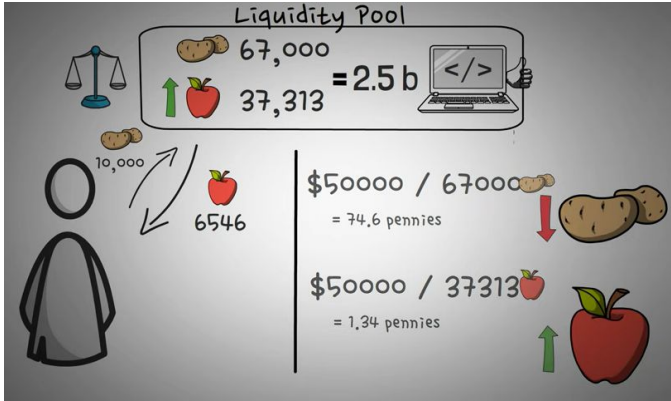
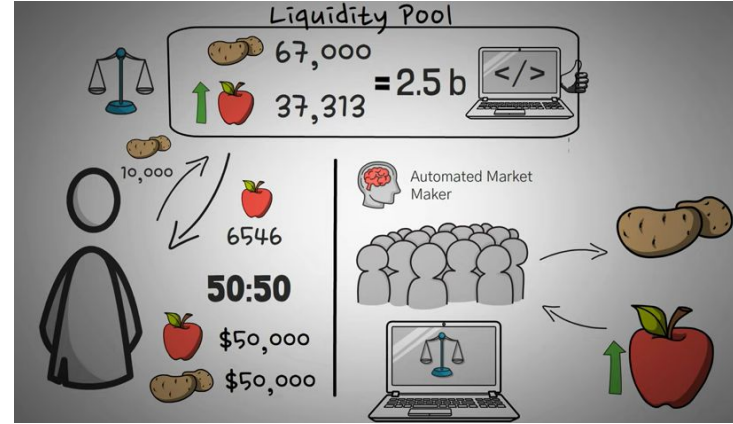
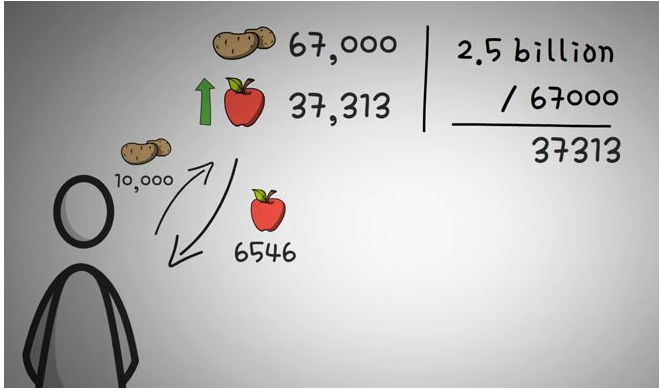
Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [YouTube video : AMM Model](#)

The DeFi Ecosystem - AMM Liquidity Pool



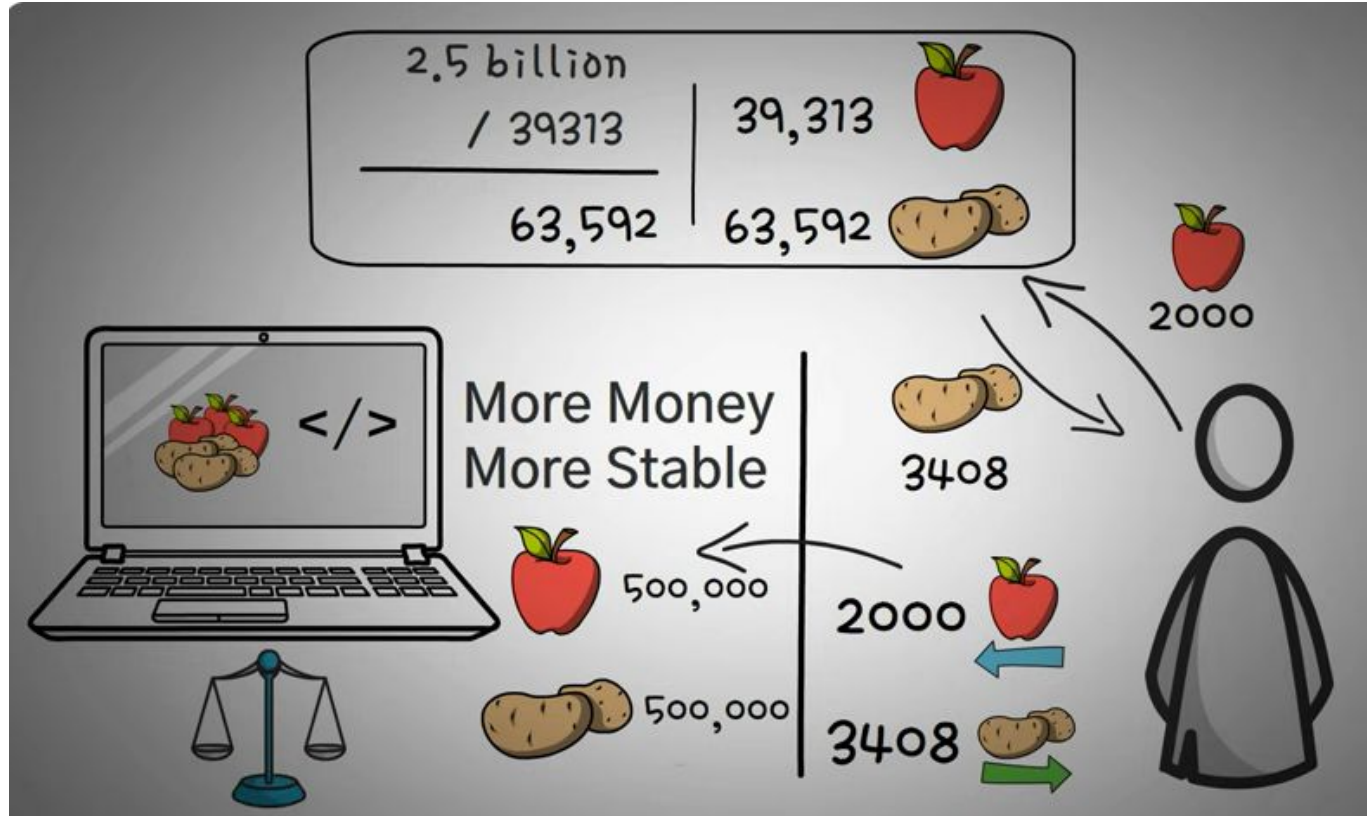
The DeFi Ecosystem - AMM Liquidity Pool



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [YouTube video : AMM Model](#)

The DeFi Ecosystem - AMM Liquidity Pool



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [YouTube video : AMM Model](#)

Trading Fees

- Pool fee: 0.3%
- User swaps ₹10,000 worth tokens
- Fee collected: ₹30
- Fee distributed to LPs proportionally

Liquidity Provider Earnings

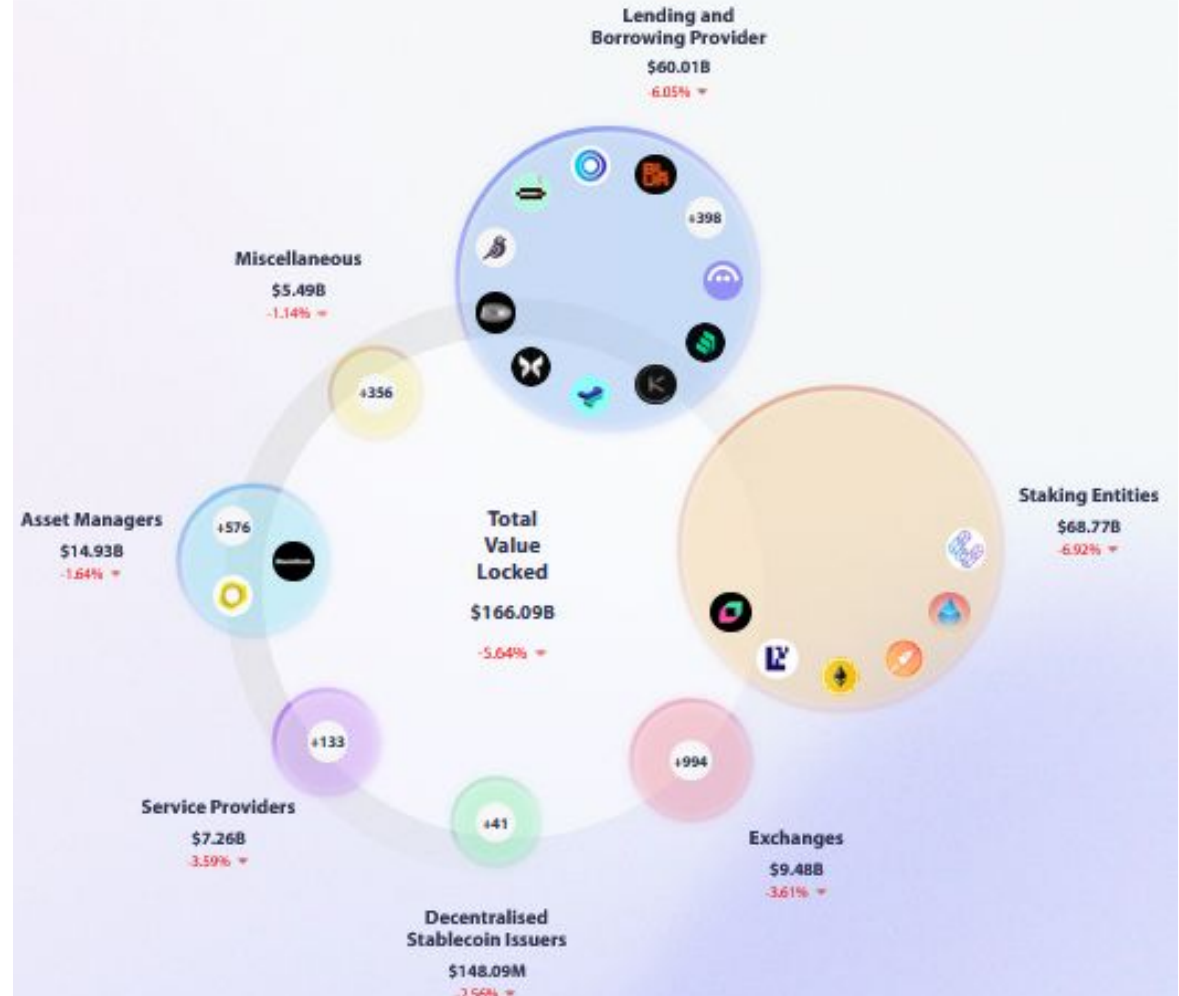
- Total pool liquidity: ₹10,00,000
- LP contribution: ₹1,00,000 (10%)
- Daily fees collected: ₹5,000
- LP earns: ₹500 per day

The DeFi Ecosystem - Users Vs Liquidity Providers

Aspect	Users	Liquidity Providers
Role	Consume liquidity	Provide liquidity
Income	Trading profits / borrowed funds	Fees + rewards
Capital Required	Low to medium	High
Risk Type	Slippage, liquidation	Impermanent loss
Example Protocol	Uniswap, Aave	Uniswap LP, Curve LP

DeFi Ecosystem

- Visualization

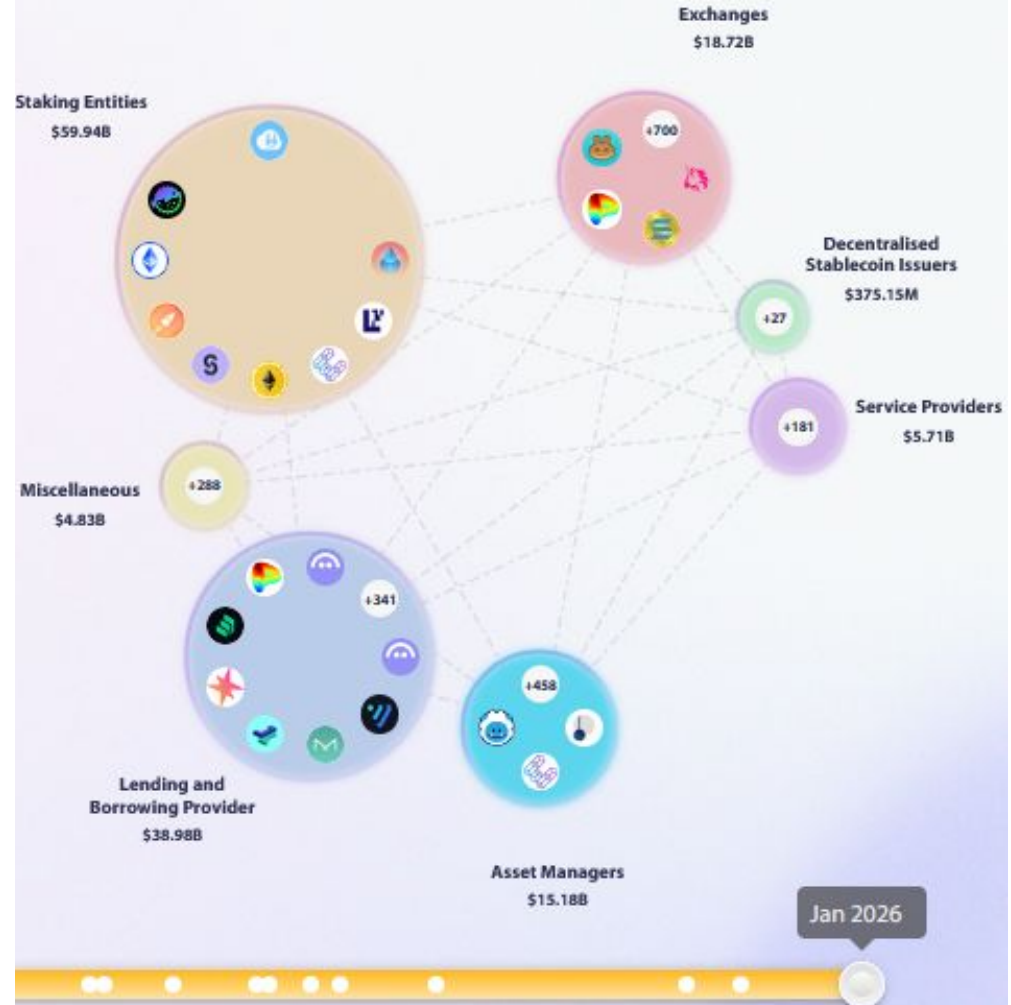


Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [University of Cambridge](#)

DeFi Ecosystem

- Value Flows



Courtesy :

- [DeFi - The Future of Finance. Campbell et. al](#)
- [University of Cambridge](#)

Applications of DeFi (Decentralized Finance)

**KYC/AML
solutions**

**Lending
Protocols**

**Data Analysis
Applications**

**Asset
Tokenization**

**Pooled
Lending**

**Creating
Stablecoins**

**Bitcoin Futures
Trading**

Courtesy :

- [DeFi - The Future of Finance, Campbell et. al](#)
- [Blockchain Simplified](#)

3. Problems That DeFi Solves

Limitations of Traditional Finance:

- Dependence on intermediaries
- Limited accessibility (bank accounts, credit checks)
- High fees & delays
- Lack of transparency
- Custodial risk

DeFi Solutions:

- Permissionless access
- Trustless transactions
- Transparency via blockchain
- Global & 24×7 availability

4. How Decentralized is DeFi?

DeFi decentralization exists on a **spectrum**, not binary.

1. Centralized

Characteristics:

- Custodial
- Centralized price feeds
- Centrally controlled interest rates

Examples:

Salt, BlockFi, Celsius

2. Semi-Decentralized DeFi

Characteristics:

- Non-custodial
- Decentralized price feeds
- Permissionless lending & liquidation
- Decentralized interest rates

Examples:

- Compound
- MakerDAO
- dYdX
- bZx



Most current DeFi applications fall in this category

How Decentralized is DeFi?

Traditional Financial System



Decentralised Financial System



How Decentralized is DeFi?

3. Fully Decentralized DeFi

Characteristics:

- Every component decentralized
- Governance via DAO
- No single point of control

Current Status:

- No DeFi protocol is fully decentralized yet

5. DeFi Key Categories – Overview

Major DeFi Categories:

1. Stablecoins
2. Lending & Borrowing
3. Exchanges
4. Derivatives
5. Fund Management
6. Lottery
7. Payments
8. Insurance

DeFi Key Categories – Overview

1. Stablecoins

- Designed to reduce crypto price volatility
- Pegged to stable assets (e.g., USD)

Types:

- **Centralized:** USDT (trust-based)
- **Decentralized:** Over-collateralized, DAO governed

Importance:

- Stable store of value
- Improves DeFi usability

2. Stablecoin Pegging

- Pegging = maintaining fixed value (e.g., 1 USD)
- Achieved using:
 - Over-collateralization
 - Smart contracts
 - Liquidation mechanisms
- Reserves are publicly auditable on blockchain

3. Lending & Borrowing

Traditional Finance Issues:

- Bank accounts required
- Credit scores & paperwork
- Excludes billions globally

DeFi Advantages:

- No credit checks
- Anyone can borrow using crypto collateral
- Earn interest by supplying liquidity

4. Decentralized Exchanges (DEXs)

Centralized Exchanges:

- Custodial
- Risk of hacks
- User funds controlled by platform

Decentralized Exchanges:

- Non-custodial
- Users retain asset control
- No trusted intermediary required

5. Derivatives

- Financial contracts derived from underlying assets
- Used for hedging and risk management

DeFi Innovation:

- On-chain derivatives
- Permissionless access
- Reduced counterparty risk

6. Fund Management

Traditional Fund Management:

- Active or passive
- Managed by institutions

DeFi Fund Management:

- Mostly passive
- Transparent strategy
- Automated via smart contracts
- Lower fees

7. Lottery (No-Loss Lottery)

- Funds pooled into smart contracts
- Capital invested in DeFi lending
- Interest rewarded to random winner
- Participants get original funds back

Eliminates custodian risk and losses

8. Insurance

Need for DeFi Insurance:

- Smart contract vulnerabilities
- Risk of hacks

DeFi Insurance:

- Covers losses from exploits
- Decentralized risk pooling
- Transparent claim mechanisms